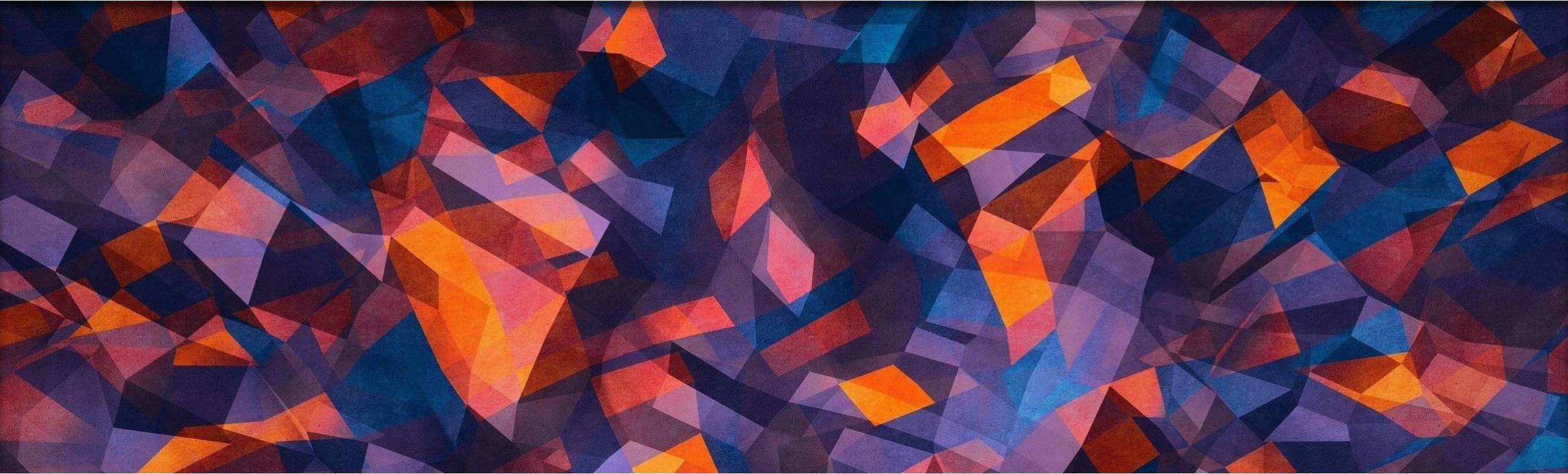




Universität St.Gallen

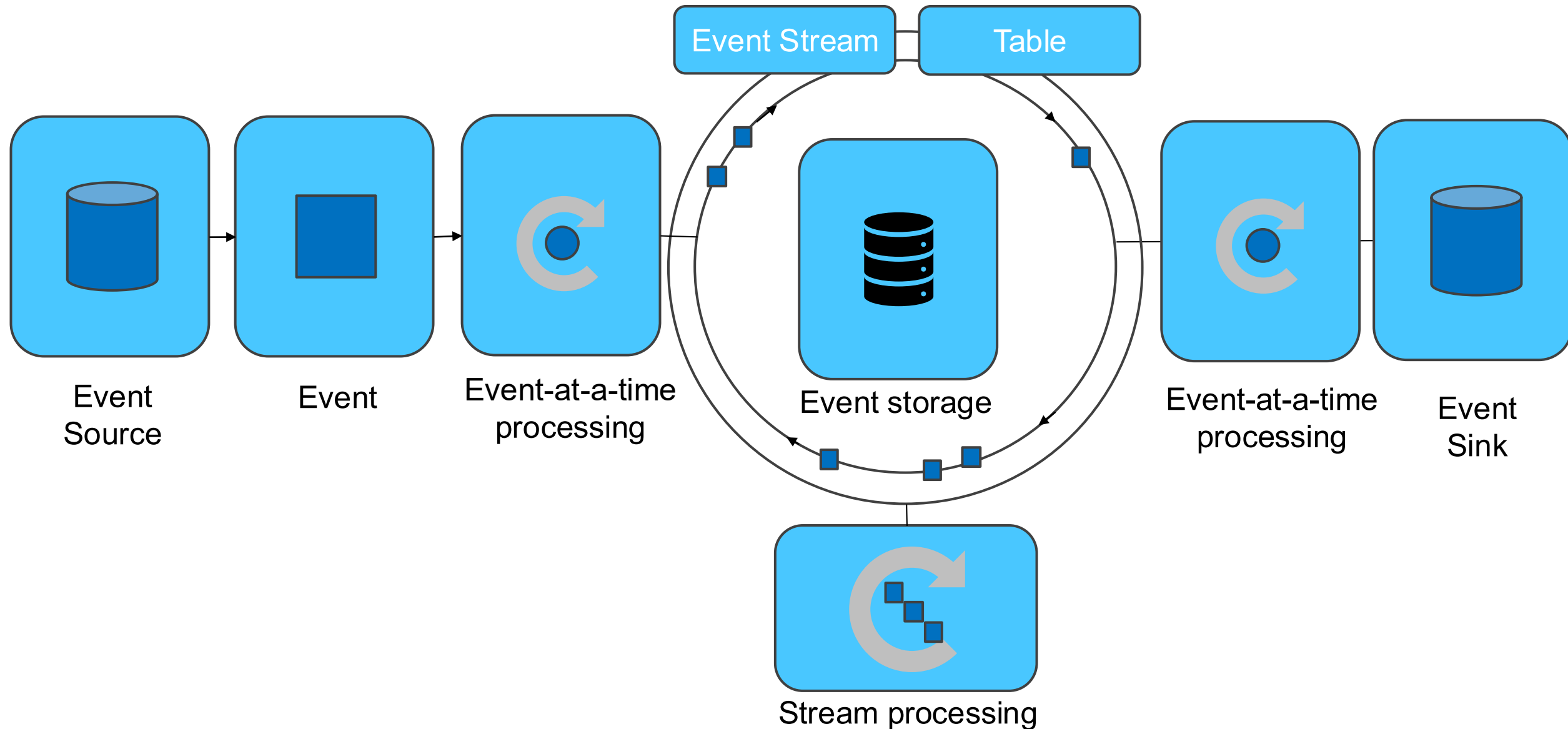


Event-driven and Process-oriented Architectures

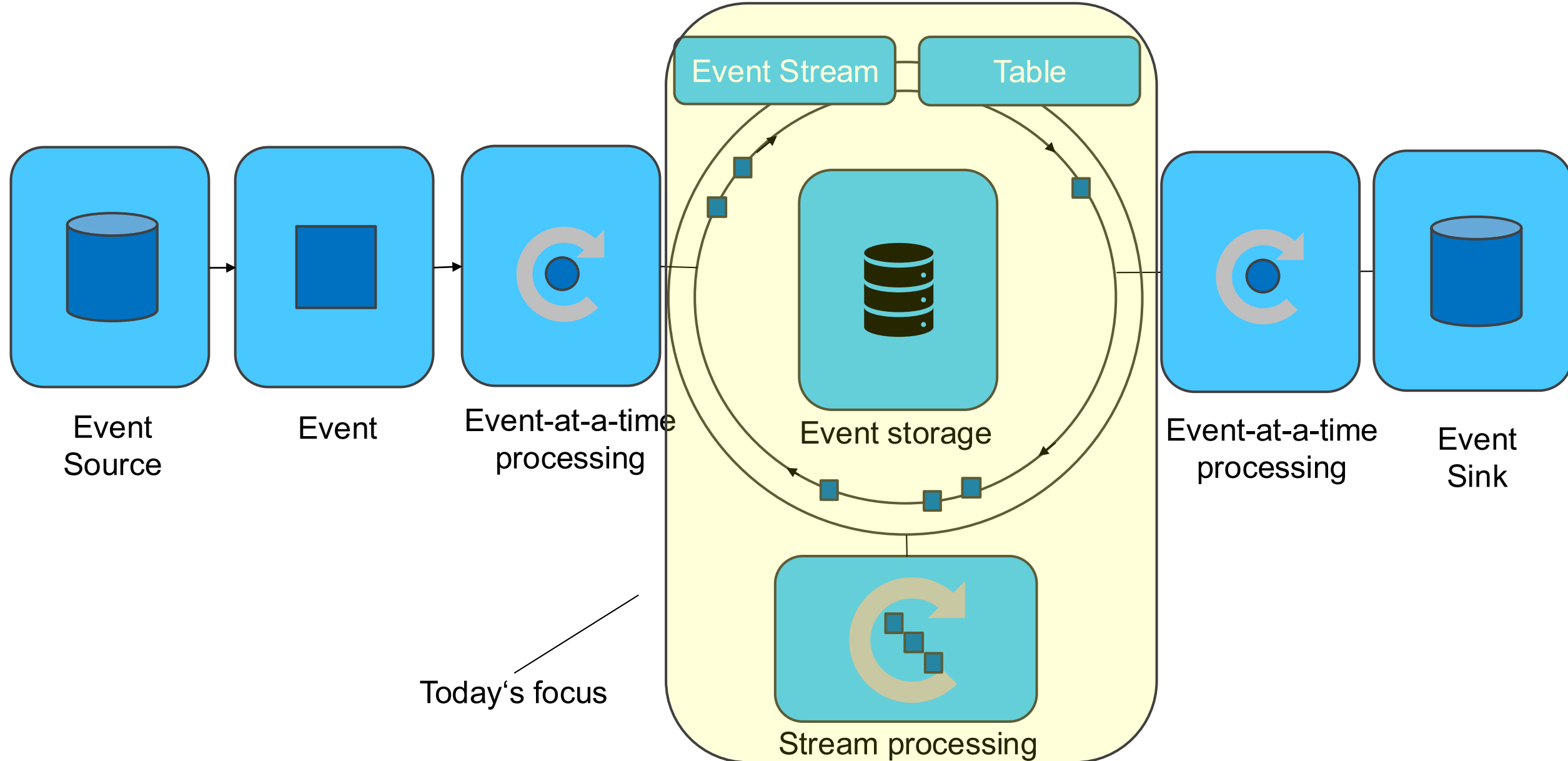
Stream Processing

Lecture 9

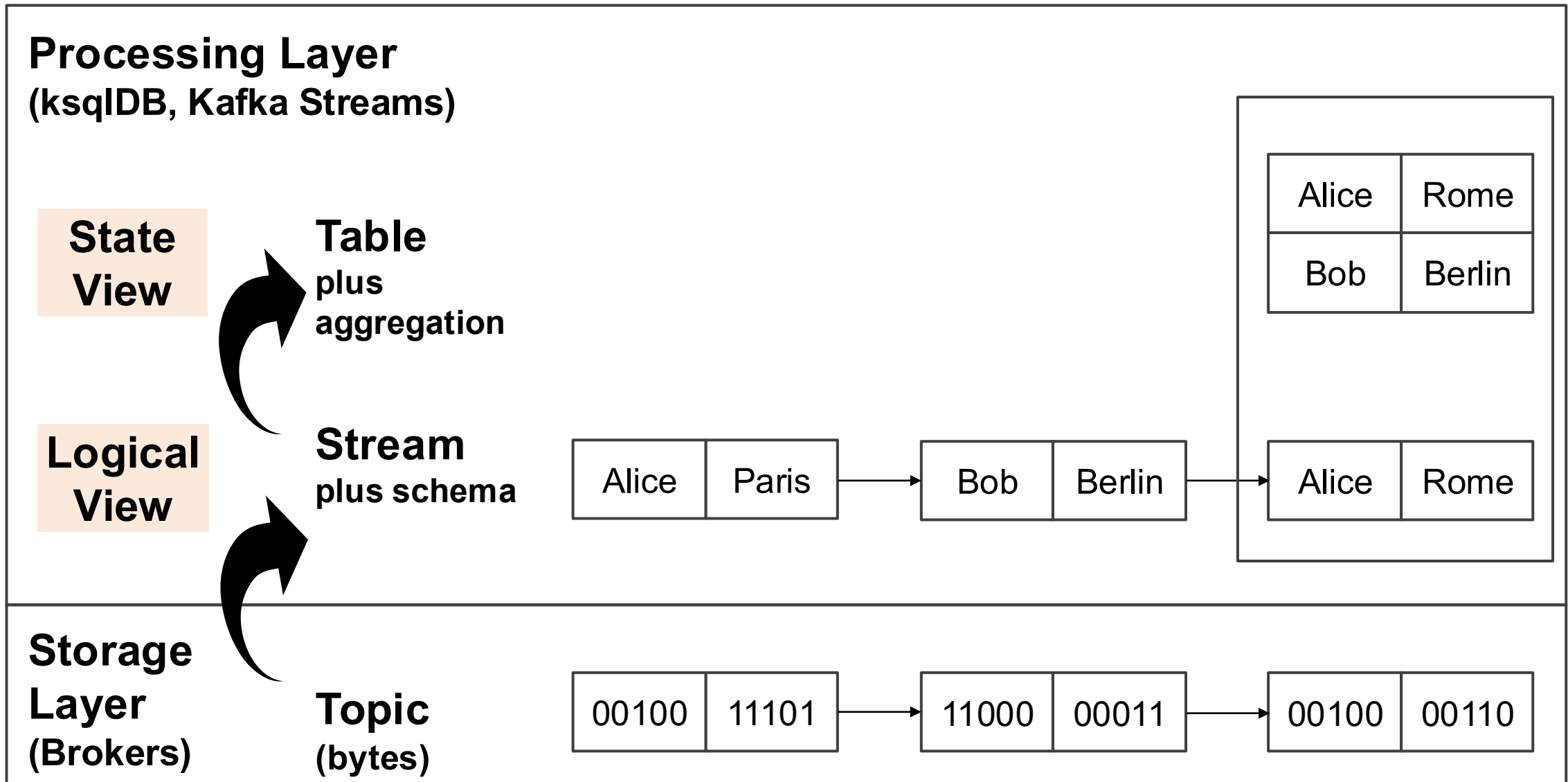
Event Streaming Platform



Event Streaming Platform



From Storage to Processing



Benefits of Stateful Processing

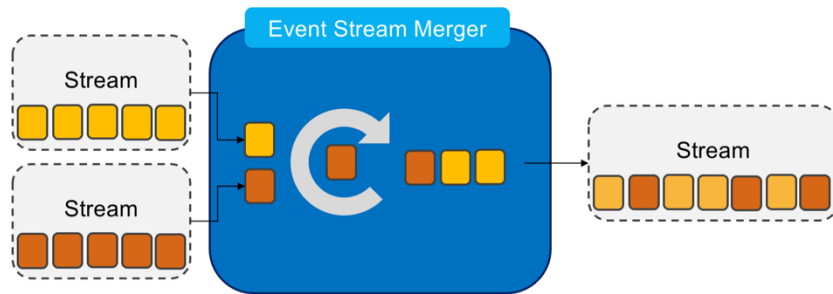
- Understand relationship between events
- Detect patterns and behaviors
- Perform aggregations and enrich data

- Events = immutable facts
 - Stateless processing = processes facts independently
 - Stateful processing = accumulates facts → captures behavior
- Enables recognizing and reacting to patterns

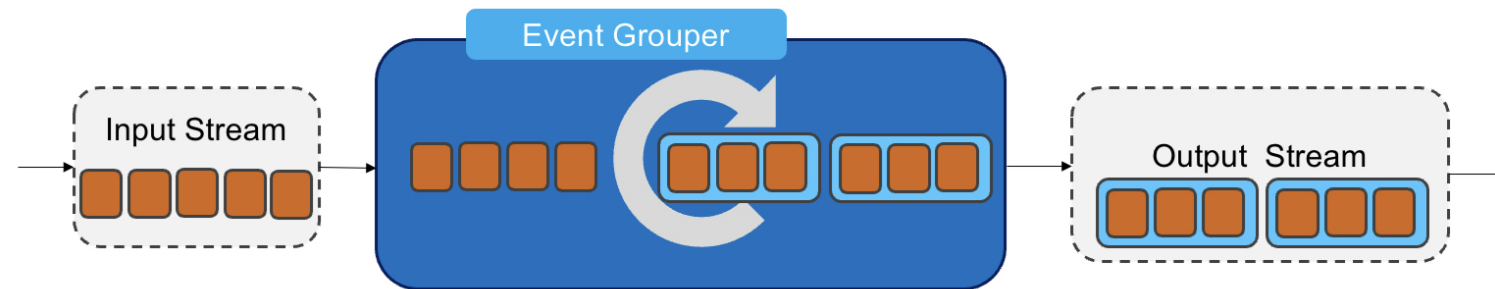
Stateful Processing Enables

- Build tables (snapshots) from streams
- Query current state in real-time
- Understand behavior from accumulated events

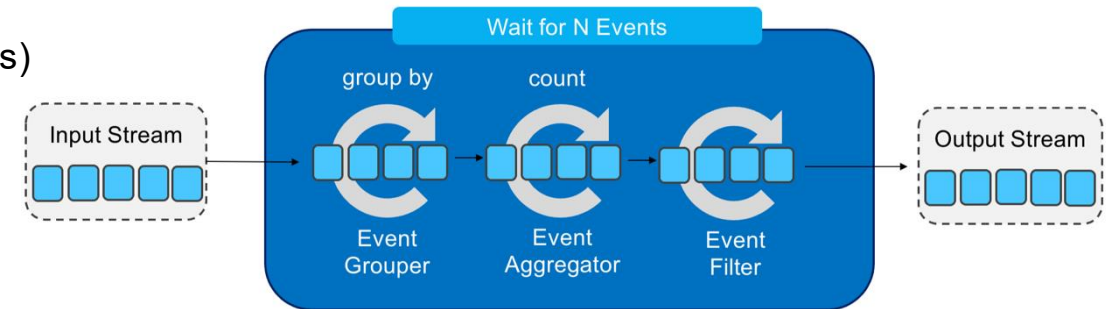
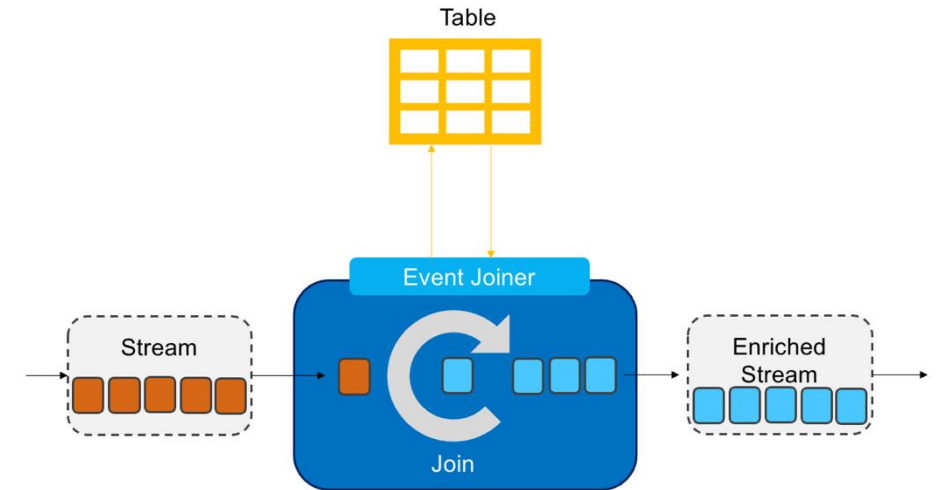
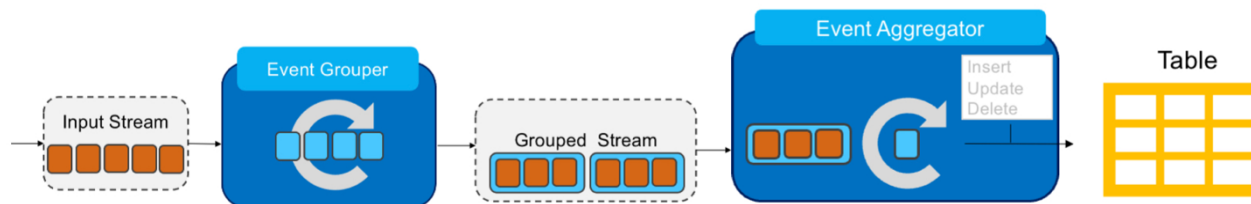
Preview (Stateful) Stream Processors



Already covered last week (stateless)



(Windowing or time-based grouping is stateful; field-based grouping is stateless)



Patterns adapted from <https://developer.confluent.io/patterns>

How State is Captured and Queried in Kafka Streams

- Stateful processing requires memory of past events
- We need a way to store and retrieve this state
- In Kafka Streams this is called a *state store*
- In Kafka Streams state stores are local and managed by the processing applications

Common Characteristics of All Default State Store Implementations

- **Embedded**

- In Kafka Streams, state stores are *embedded* (local to each instance)
- Why?
 - Low latency
 - No shared state → no contention
- Trade-off
 - State must be replicated (via Kafka) for fault tolerance
- Implementation
 - Uses RocksDB (fast, embedded key-value store)

Common Characteristics of All Default State Store Implementations

- Two types: **persistent** versus **in-memory**
- **Persistent:**
 - Spill state to disk (not only memory)
 - On failure:
 - Restore only missing updates from Kafka (changelog) → Faster recovery than rebuilding everything from scratch

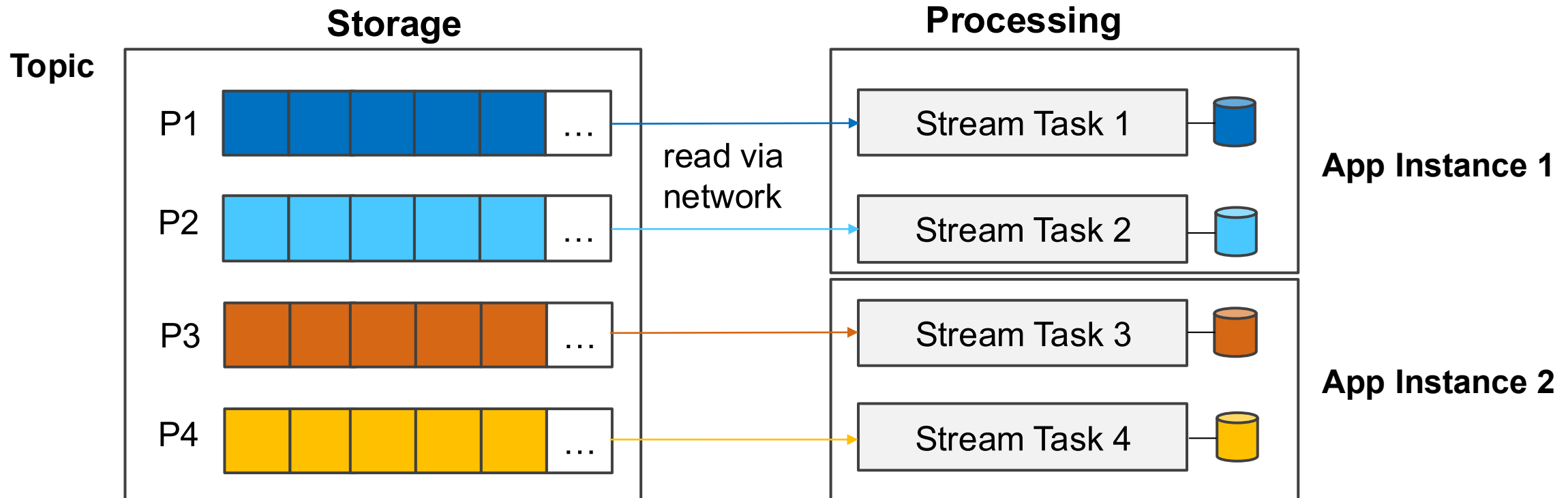
Common Characteristics of All Default State Store Implementations

- **Fault Tolerance**

- State stores are backed by Kafka changelog topics
- On failure: state is rebuilt by replaying the changelog
- Optional: standby replicates for faster recovery

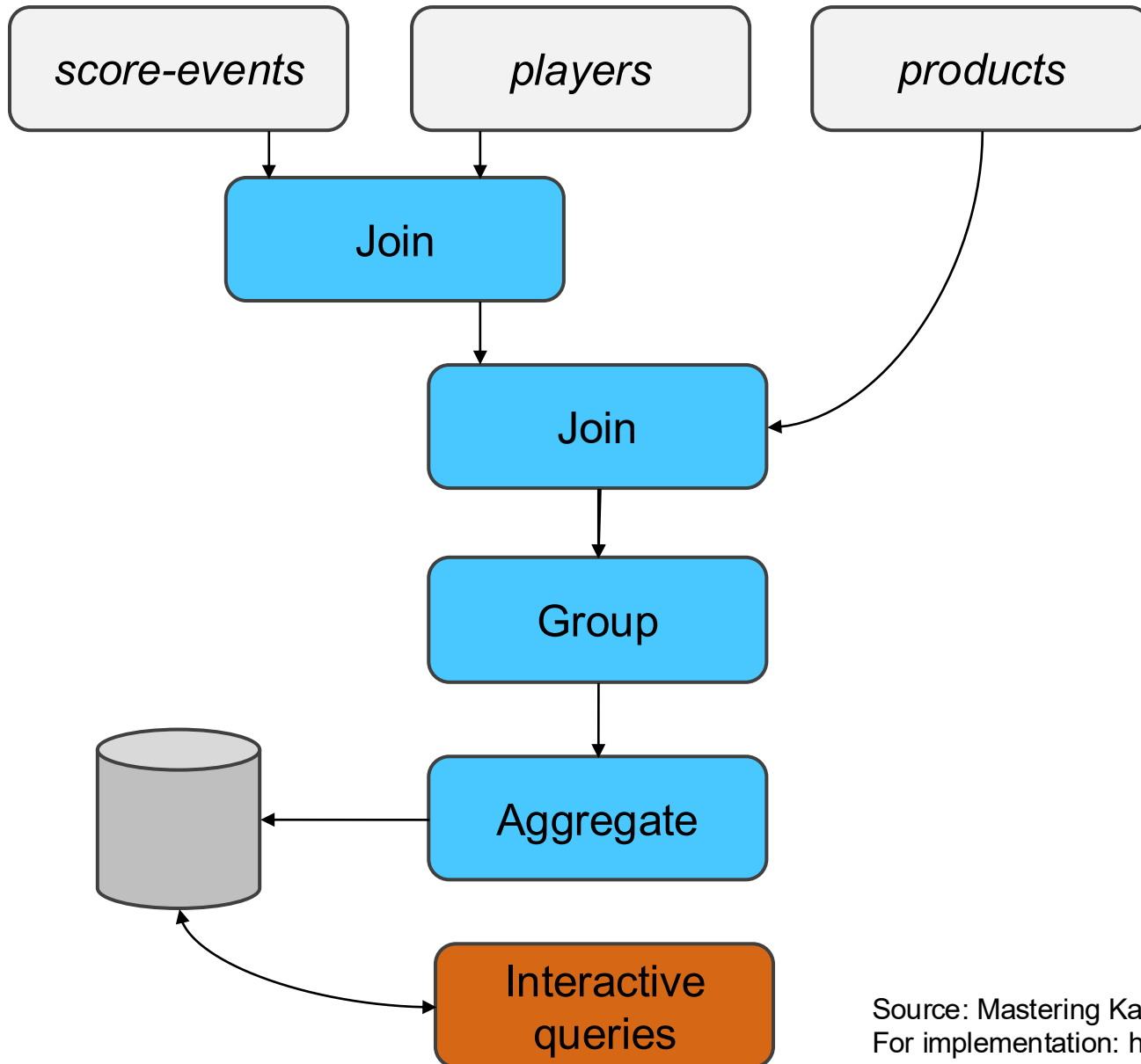
Streams/Tables are Partitioned as is their Processing

- Partitions are processed independently (parallelism)
- Each partition → one task
- Each task maintains its own local state (state store)
- → State is partitioned the same way as the data

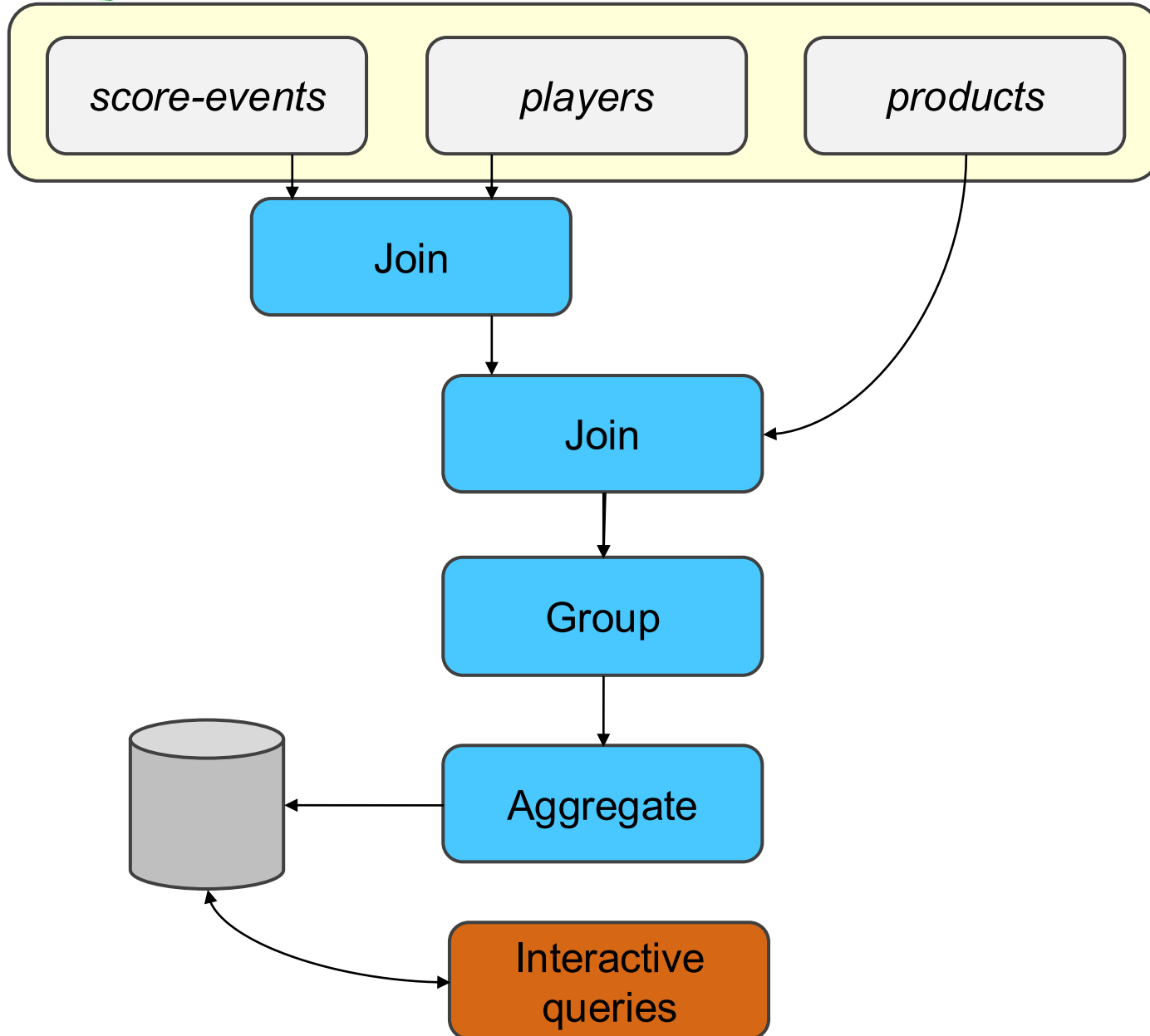


- Two important abstractions in Kafka Streams
 - **KTables:**
 - Represents changing state (latest value per key)
 - Partitioned (distributed)
 - **GlobalKTables:**
 - Same data on every instance
 - fully replicated
- Note: ksqlDB at the moment does not support Global Tables

Example Topology Stateful Processing



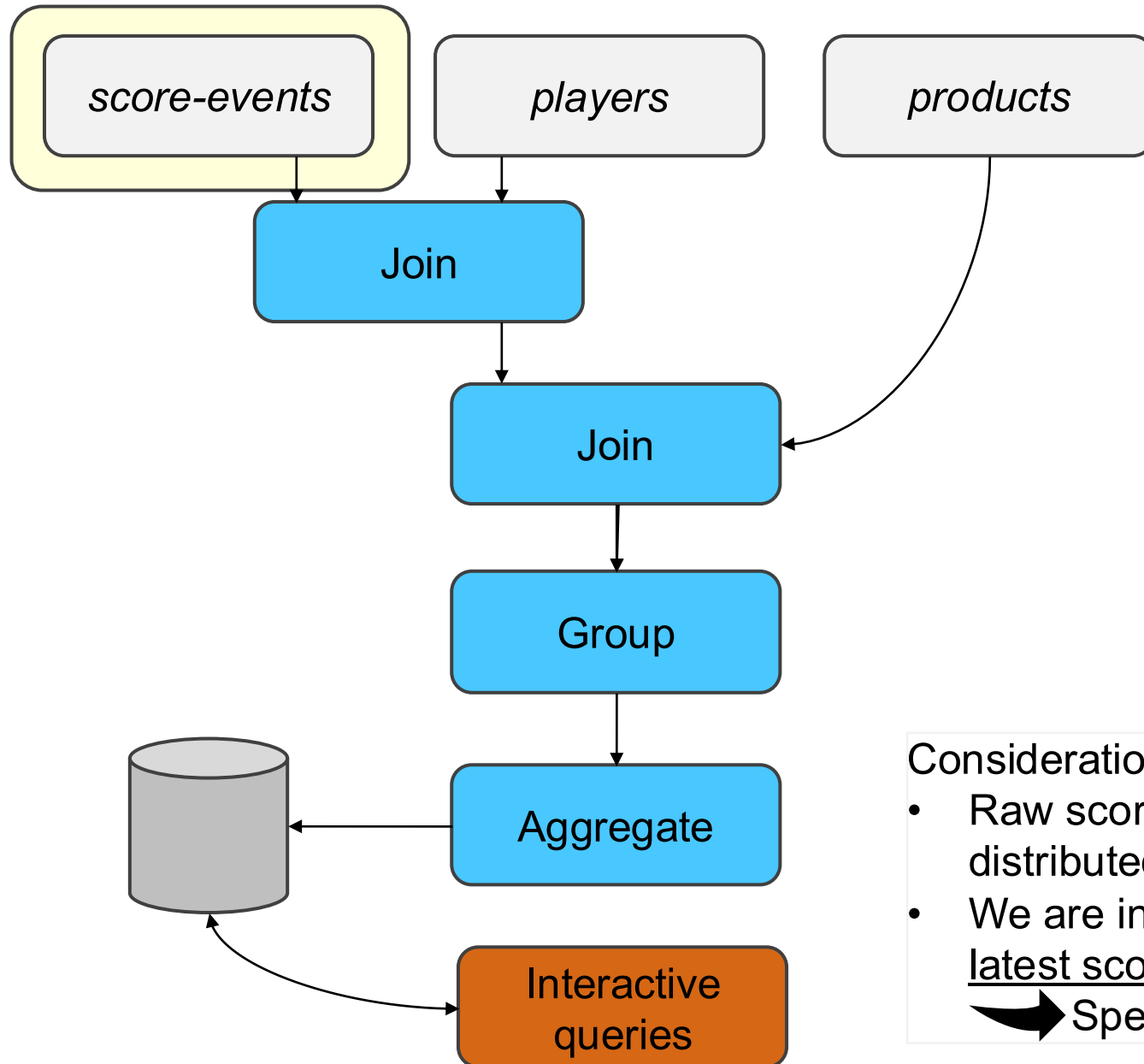
Source: Mastering Kafka Streams and ksqlDB
For implementation: <https://github.com/mitch-seymour/mastering-kafka-streams-and-ksqldb>



Example Topology

Stateful Processing

- Which abstractions should we use for our source processors?
 - KStream?
 - KTable
 - KGlobalTable?



Example Topology

Stateful Processing

- Which abstractions should we use for *score-events*?

Example record

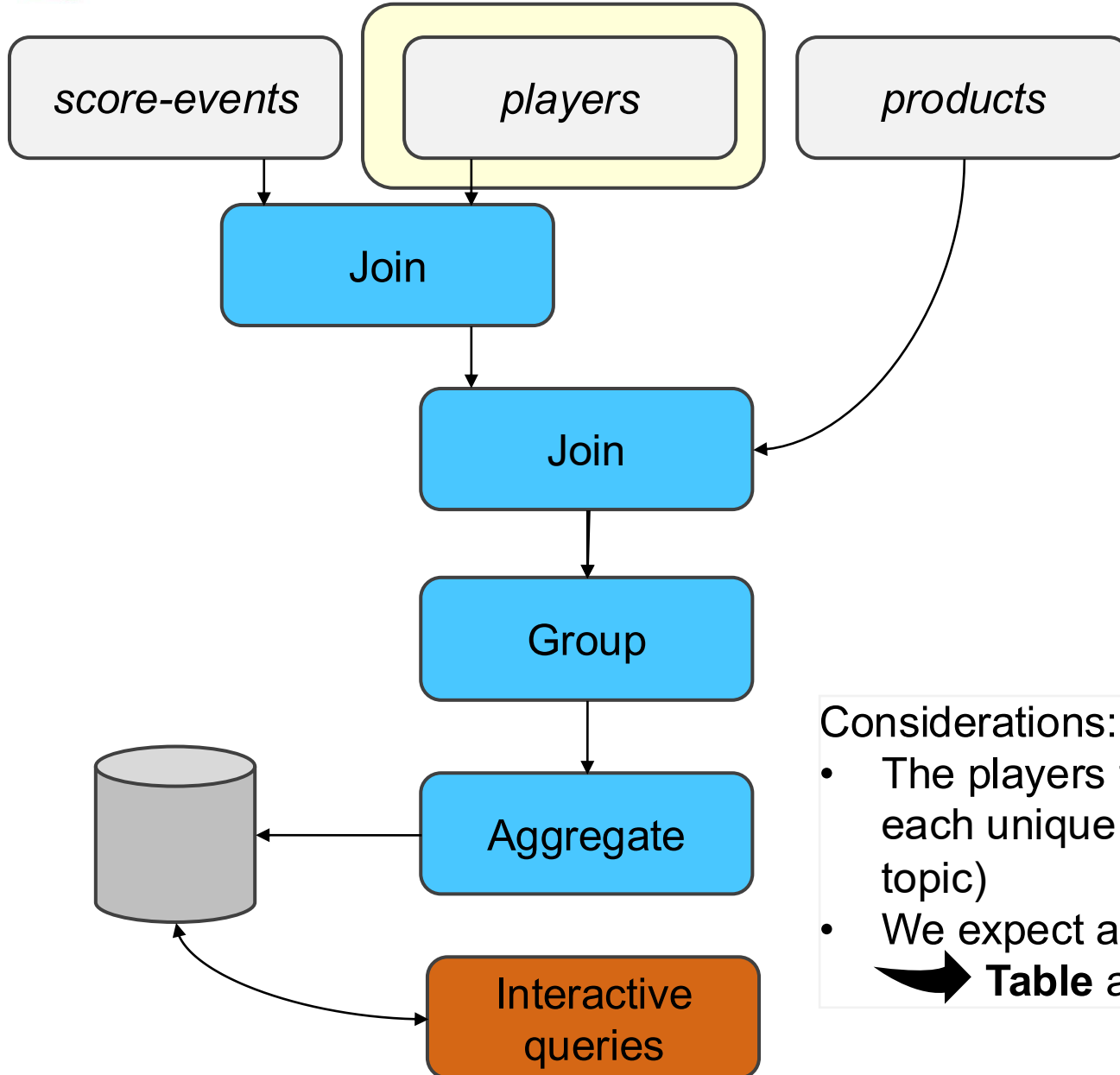
```
{
  "score": 422,
  "product_id": 6,
  "player_id": 1
}
```

Considerations:

- Raw score events are not keyed and therefore distributed round-robin in an uncompacted topic
- We are interested in the highest score and not the latest score



Speaks for using a **stream** abstraction



Example Topology

Stateful Processing

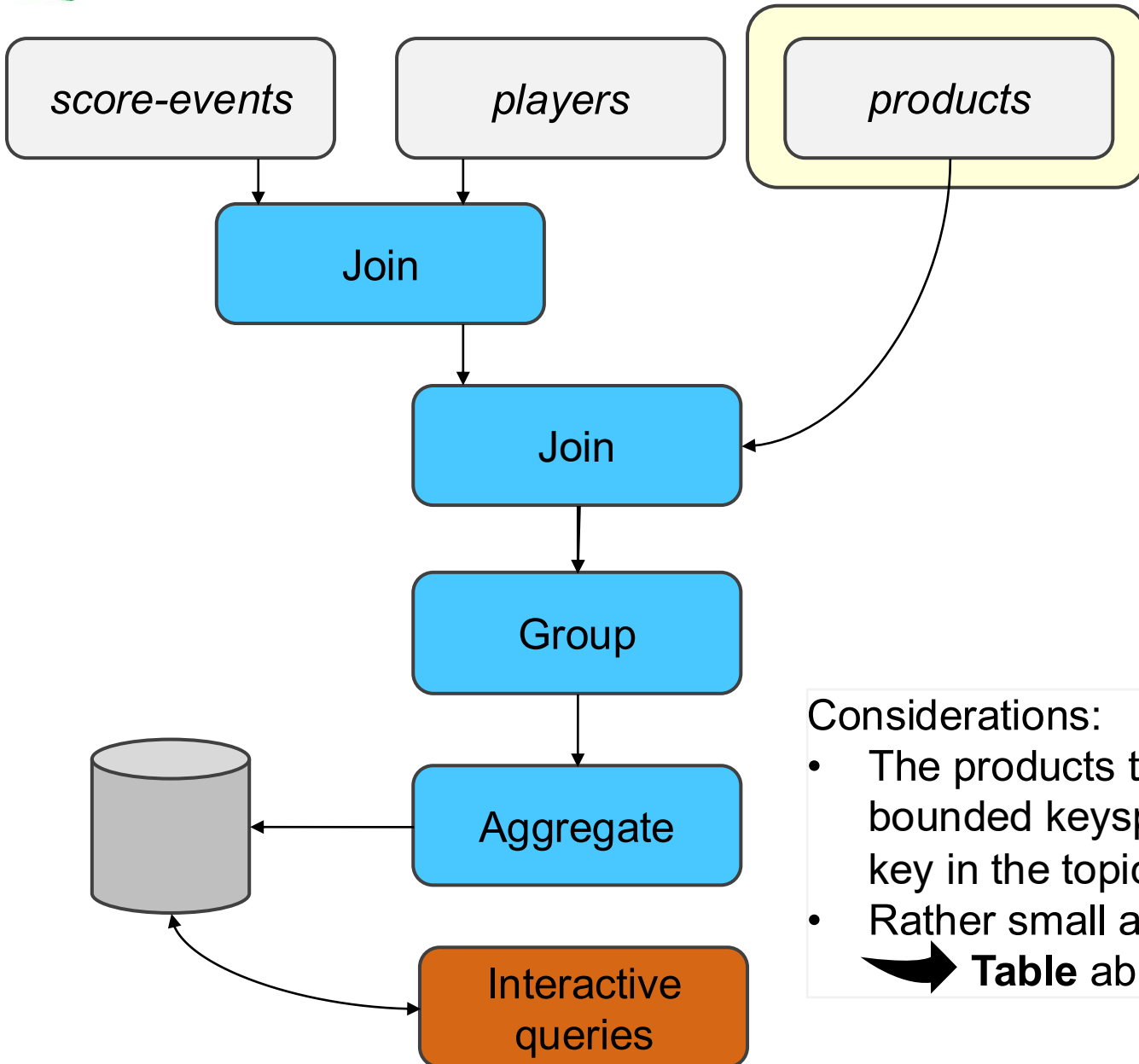
- Which abstractions should we use for *players*?

Example record

```
{
    "id": 2,
    "name": "Mitch"
}
```

Considerations:

- The *players* topic is keyed and only the latest record for each unique key in the topic is relevant (compacted topic)
- We expect a potentially large keyspace
➡ **Table** abstraction (KTable)



Example Topology

Stateful Processing

- Which abstractions should we use for *products*?

Example record

```
{
  "id": 1,
  "name": "Super Smash Bros"
}
```

Considerations:

- The *products* topic is keyed. It is compacted and has a bounded keyspace - only latest record for each unique key in the topic is relevant; limited amount of products
- Rather small and static keyspace

➡ **Table** abstraction (GlobalKTable)

- **KTable:**

- Partitioned state (distributed across tasks)
- Scales with data size
- Used for large, evolving datasets

- **GlobalKTable:**

- Fully replicated state (available on every task)
- No partitioning needed for joins
- Used for small, static reference data

Streams vs. Tables vs. Global Tables

Concept	Schema	Partitioned	Unbounded	Ordering Guarantee***	Mutable	Single Value Per Key	Fault Tolerant
Storage Layer							
Topic	No* (Raw Bytes)	Yes	Yes	Yes	No	No	Yes
Processing Layer							
Stream	Yes	Yes	Yes	Yes	No	No	Yes
Table	Yes	Yes	No**	No	Yes	Yes	Yes
Global Table	Yes	No	No**	No	Yes	Yes	Yes

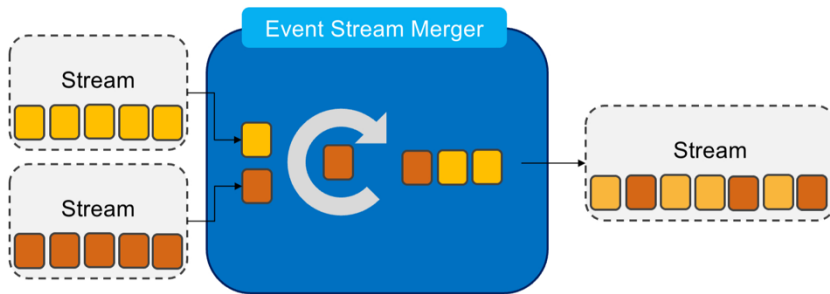
*When using Schema Registry with Avro, then a schema is technically available for the data in a topic.

**Generally speaking, the answer is yes but, in practice, tables are almost always bounded due to finite key space.

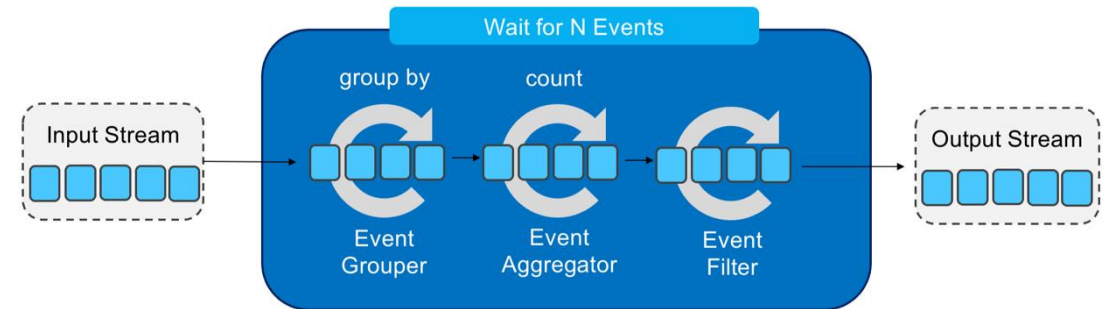
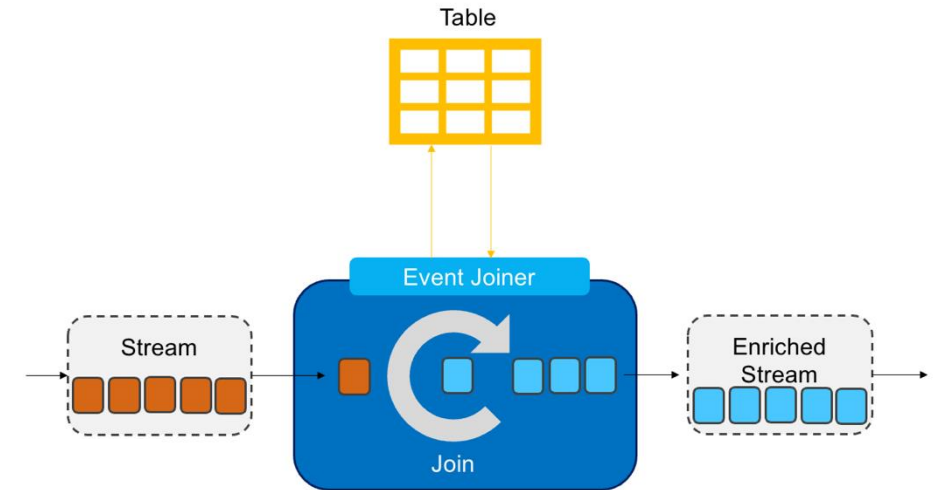
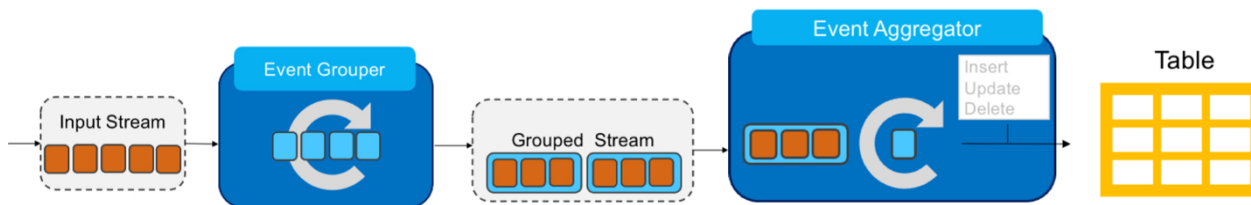
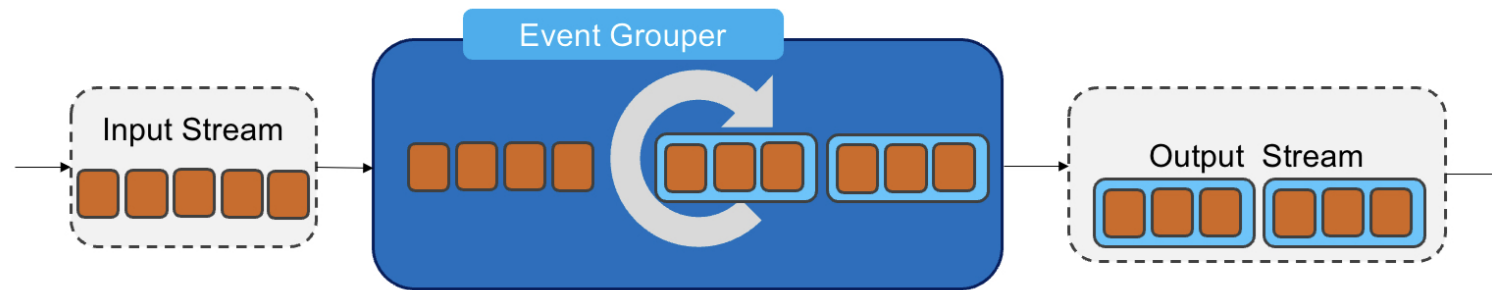
***Ordering per partition, not across the multiple partitions in a topic.

Source: <https://www.confluent.io/blog/kafka-streams-tables-part-3-event-processing-fundamentals/>

Selected Types of Stream Processors



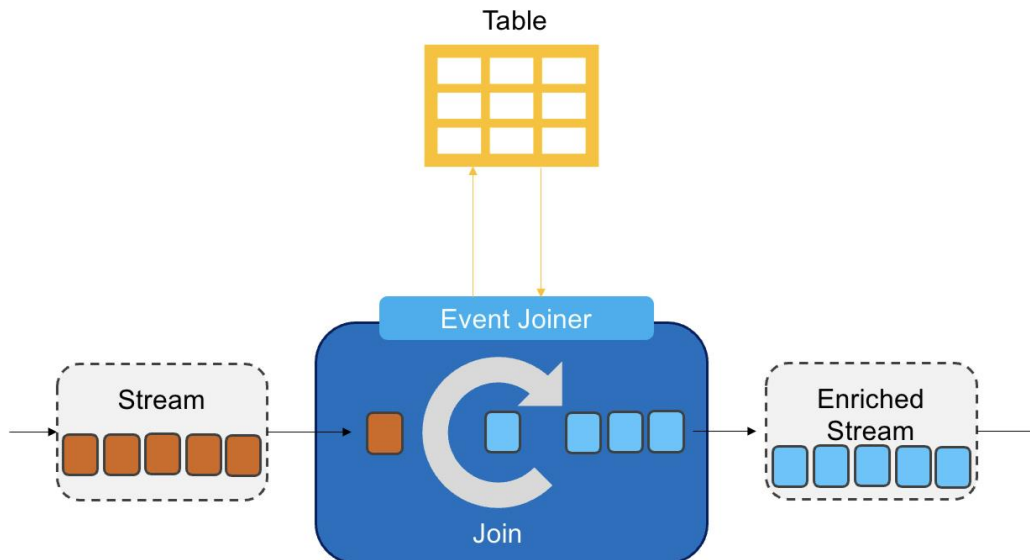
Already covered last week (stateless)



Patterns adapted from <https://developer.confluent.io/patterns>

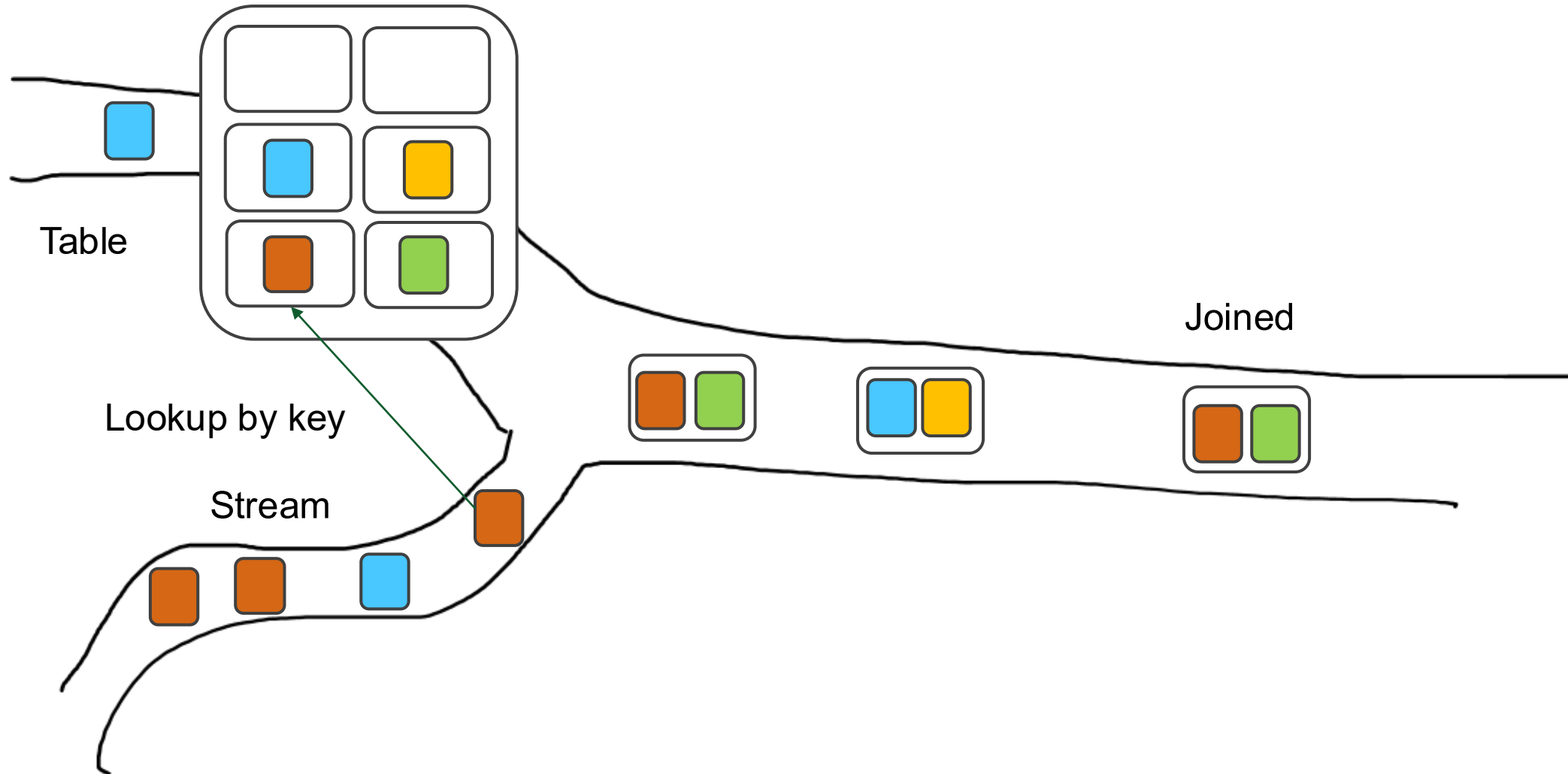
Event Joiner. Event Streams may need to be joined (i.e., enriched) with a Table or an Event Stream to provide more comprehensive details about an event

- Events in a stream are combined with a Table or another Event Stream resulting into one output stream
- The join is based on a key shared between the input stream and the other stream / table
- When joining two streams a window buffering mechanism is needed



Further information on [Event Joiner](#)
Tutorial: [Join a stream to a table](#)

Example of an Event Joiner



Join Operators (Reference)

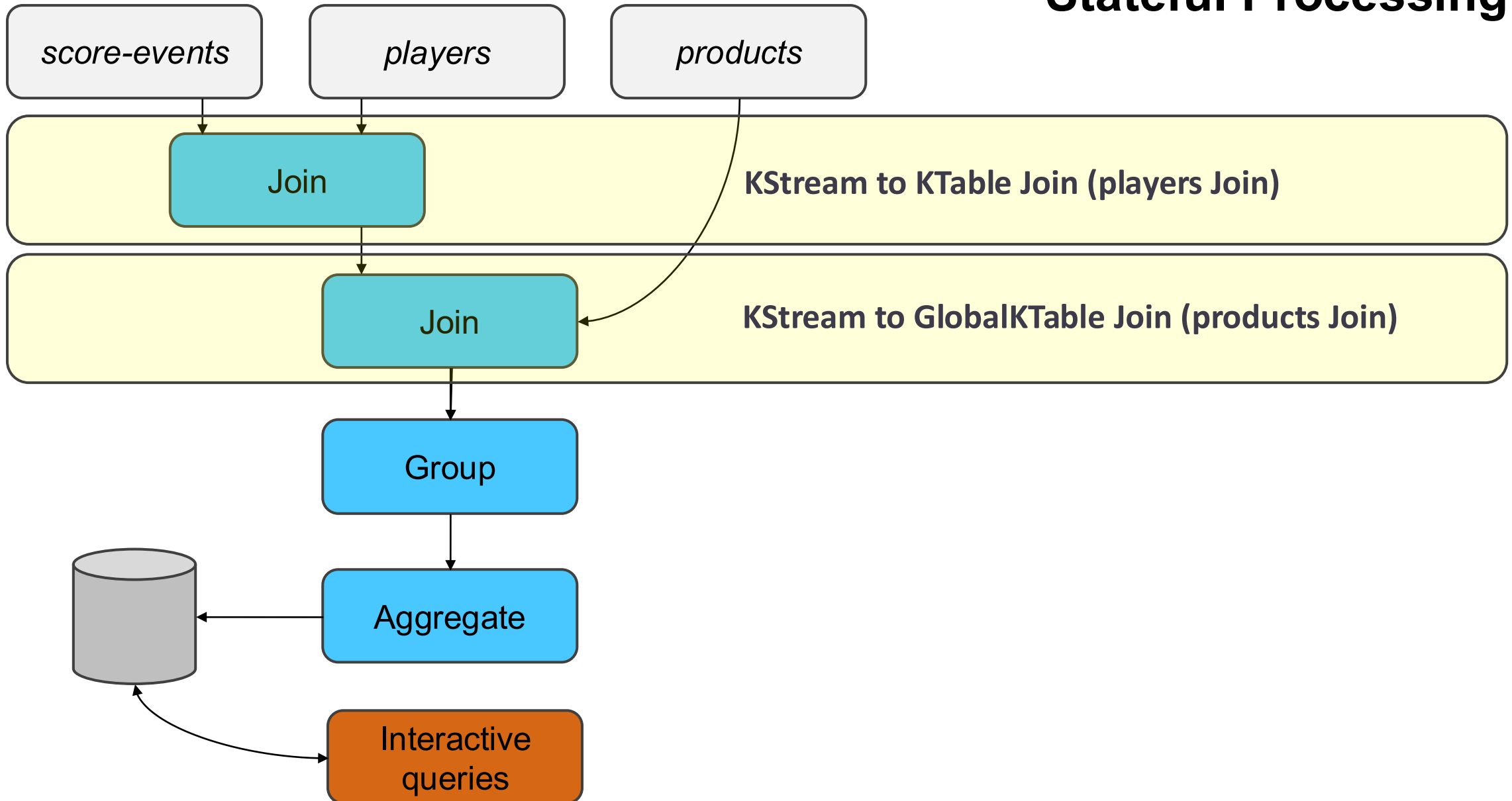
Same semantics as SQL joins — details for reference

Operator	Description
join	Inner join. The join is triggered when the input records on both sides of the join share the same key.
leftJoin	Left join. The join semantics are different depending on the type of join: • For stream-table joins: a join is triggered when a record on the <i>left side</i> of the join is received. If there is no record with the same key on the right side of the join, then the right value is set to null. • For stream-stream and table-table joins: same semantics as a stream-stream left join, except an input on the right side of the join can also trigger a lookup. If the right side triggers the join and there is no matching key on the left side, then the join will not produce a result.
outerJoin	Outer join. The join is triggered when a record on <i>either side</i> of the join is received. If there is no matching record with the same key on the opposite side of the join, then the corresponding value is set to null.

Join Types (Reference)

Type	Windowed	Operators	Co-partitioning required
KStream-KStream	Yes	<code>join</code> <code>leftJoin</code> <code>outerJoin</code>	Yes
KTable-KTable	No	<code>join</code> <code>leftJoin</code> <code>outerJoin</code>	Yes
KStream-KTable	No	<code>join</code> <code>leftJoin</code>	Yes
KStream-KGlobalTable	No	<code>join</code> <code>leftJoin</code>	No

Example Topology Stateful Processing

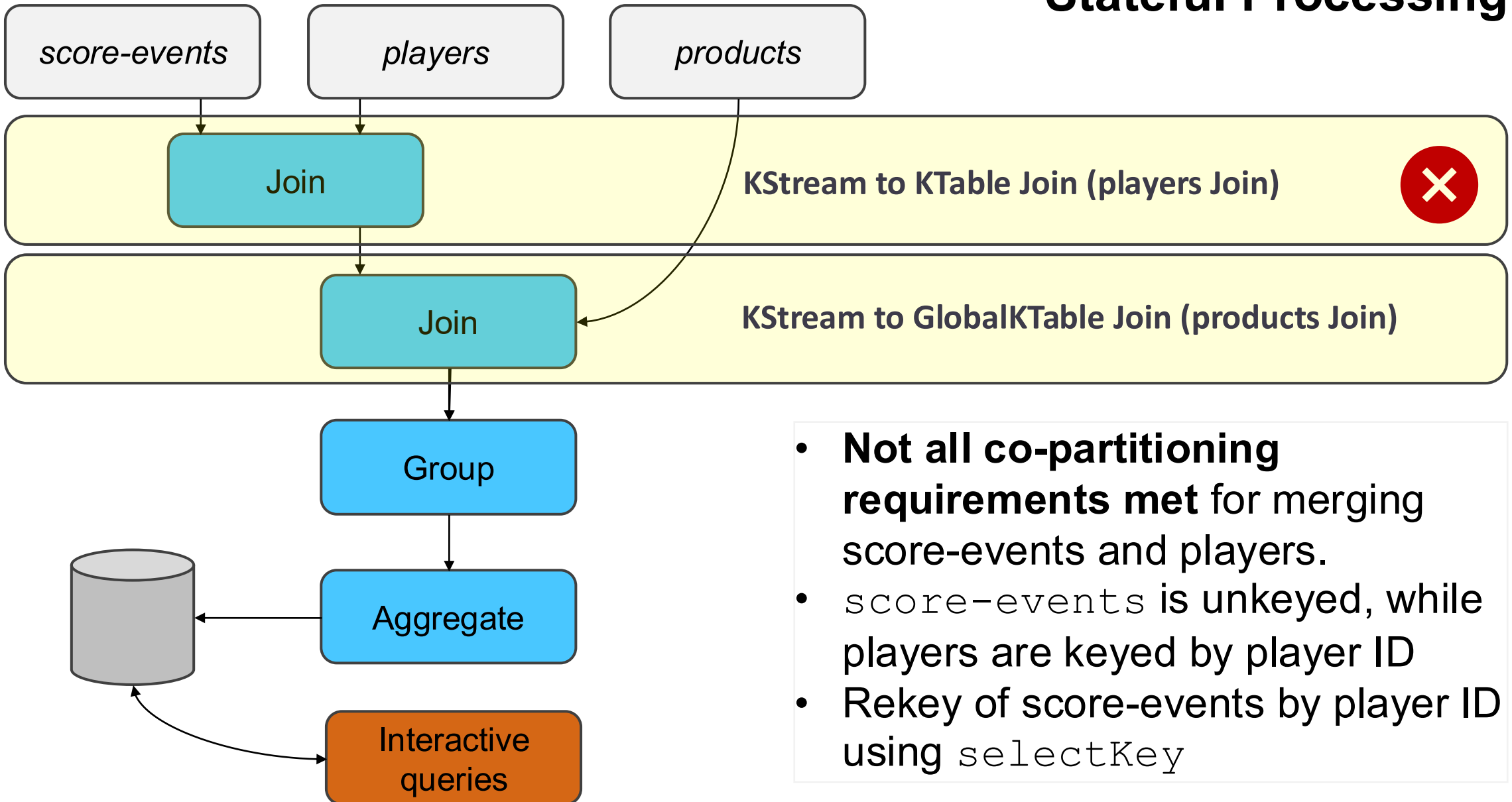


Co-partitioning (for joins)

- Related events must be in the same partition
 - Requires
 1. Same keys
 2. Same partitioning strategy
 3. Same number of partitions
- Otherwise: joins won't work correctly

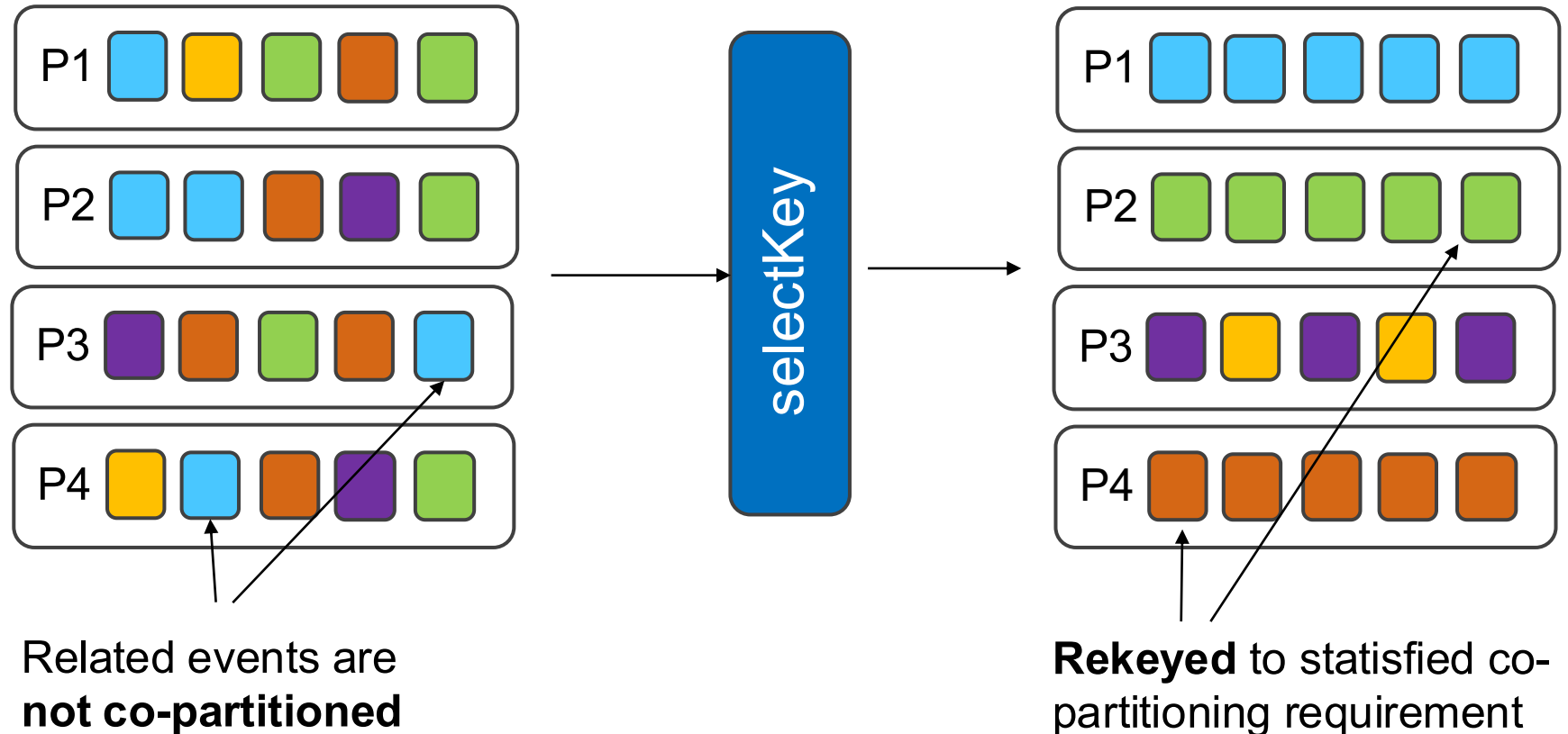
Example Topology

Stateful Processing



Rekeying Events to Satisfy Co-partitioning

- Using `selectKey` to align keys accross keys
- Triggers re-partitioning (data shuffle)
- → Trade-off: required for correctness, but expensive



Rekeying Events to Satisfy Co-partitioning

- `score-events` is unkeyed, players keyed by `playerId`
- Use `selectKey` align keys
- → Enables join (co-partitioning satisfied)

```
KStream<String, ScoreEvent> scoreEvents =  
    builder  
        .stream(  
            "score-events",  
            Consumed.with(Serdes.ByteArray(), JsonSerdes.ScoreEvent()))  
        .selectKey((k, v) -> v.getPlayerId().toString());
```

- SQL
 - SELECT defines structure of the join result
- Kafka Streams
 - ValueJoiner defines how values from both sides are combined
- Two approaches:
 - Wrapper class (keep full objects)
 - Property extraction (create new object)

Value Joiner Using Wrapper Class

```
public class ScoreWithPlayer
{
    private ScoreEvent scoreEvent;
    private Player player;

    public ScoreWithPlayer(ScoreEvent scoreEvent, Player player)
    {
        this.scoreEvent = scoreEvent;
        this.player = player;
    }
    ...
}
```

1

Data Class for wrapping the values involved in the join.

Constructor has parameter for each side of the join

Reference to each record involved in the join is saved inside the wrapper class

ValueJoiner for combining *score-events* and *players*

```
ValueJoiner<ScoreEvent, Player, ScoreWithPlayer> scorePlayerJoiner =
    (score, player) -> new ScoreWithPlayer(score, player);
```

2

Value Joiner – Property Extraction

```
public class Enriched {
    private Long playerId;
    private Long productId;
    private String playerName;
    private String gameName;
    private Double score;
    public Enriched(ScoreWithPlayer scoreEventWithPlayer, Product product) {
        this.playerId = scoreEventWithPlayer.getPlayer().getId();
        this.productId = product.getId();
        this.playerName = scoreEventWithPlayer.getPlayer().getName();
        this.gameName = product.getName();
        this.score = scoreEventWithPlayer.getScoreEvent().getScore();
    }
    ...
}
```

1

Properties from each side of the join are extracted and saved in class properties

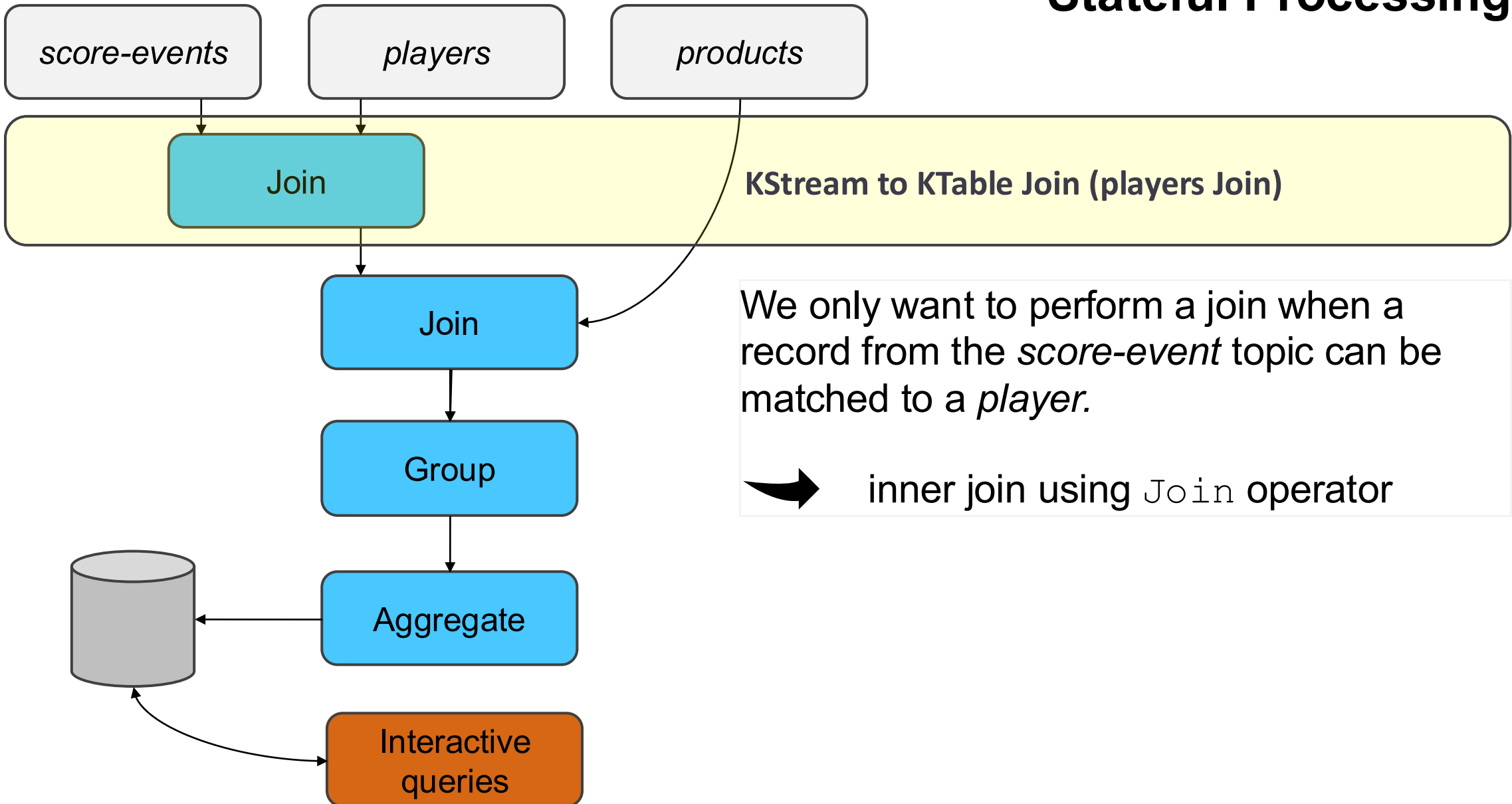
ValueJoiner for combining *players* and *products*

```
ValueJoiner<ScoreWithPlayer, Product, Enriched> productJoiner =
    (scoreWithPlayer, product) -> new Enriched(scoreWithPlayer, product);
```

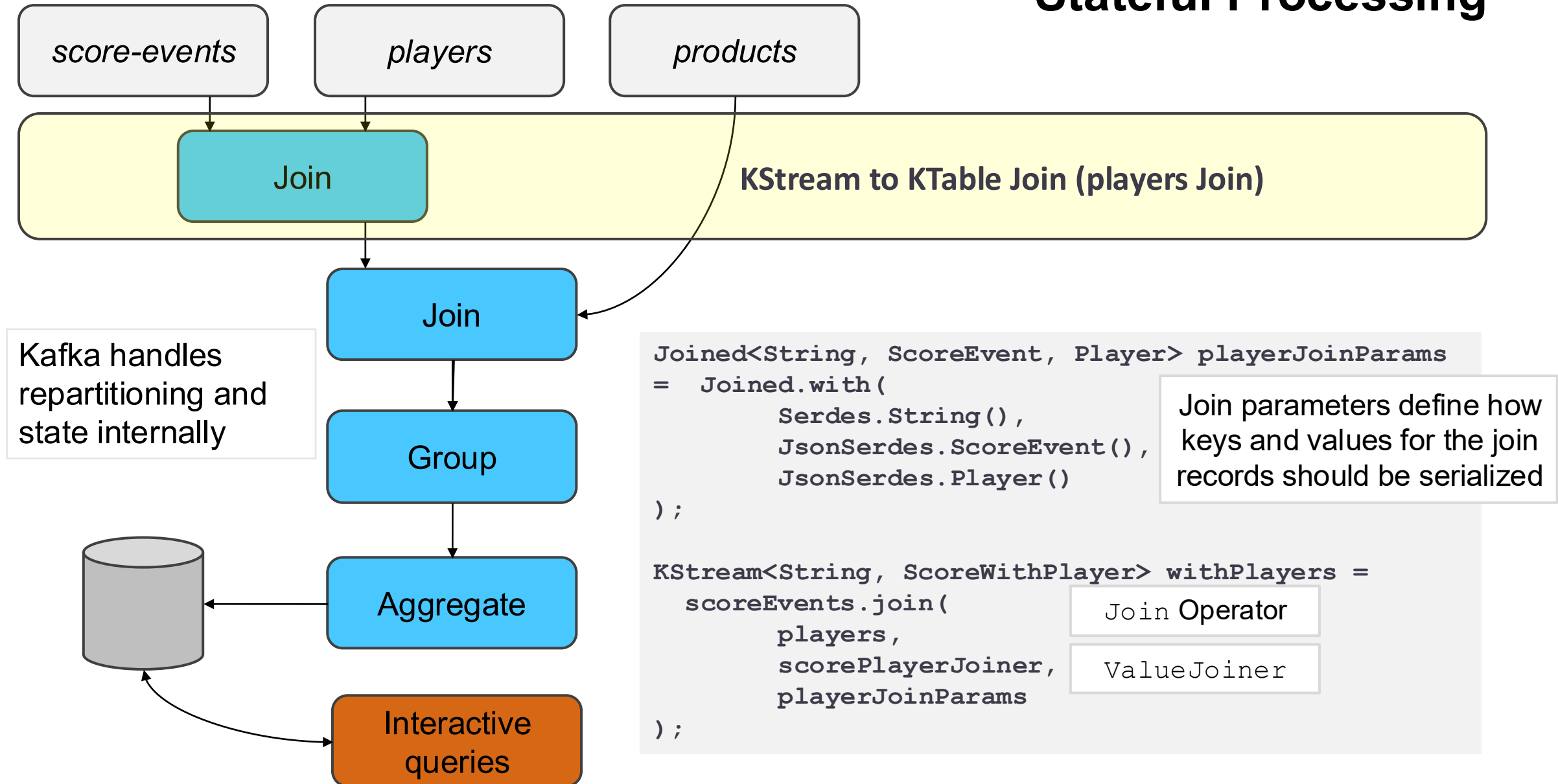
2

Example Topology

Stateful Processing

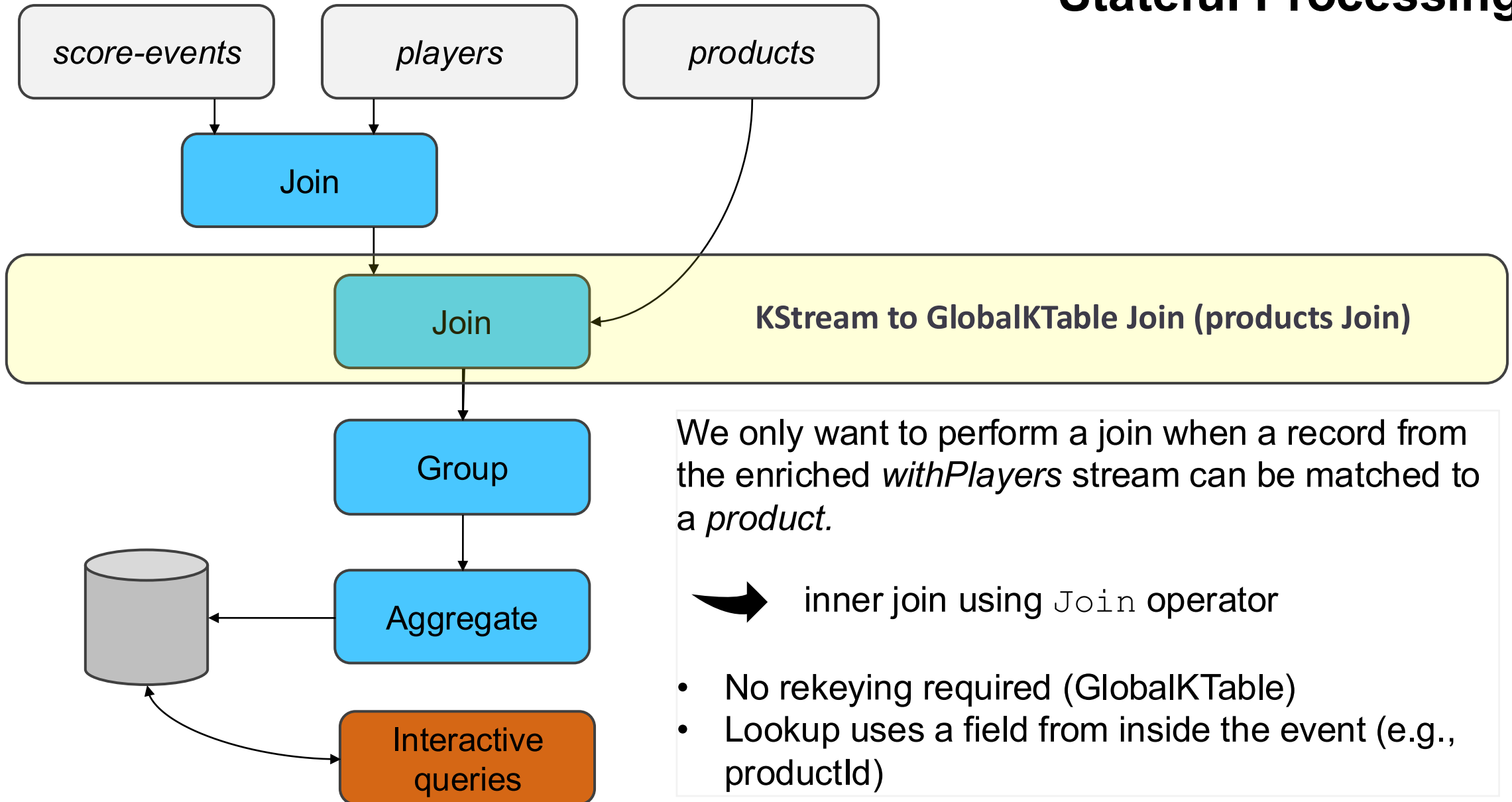


Example Topology Stateful Processing



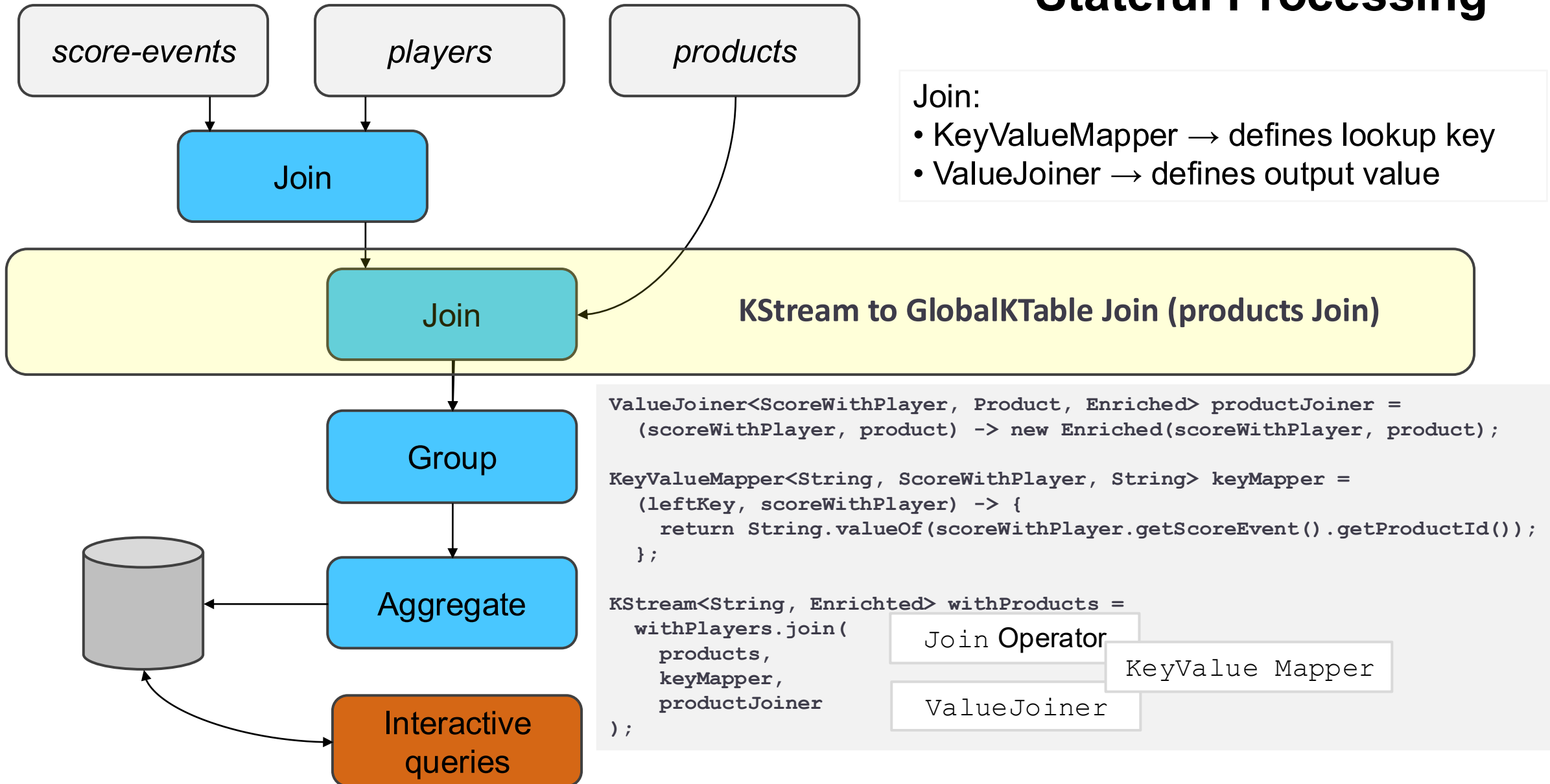
Example Topology

Stateful Processing



Example Topology

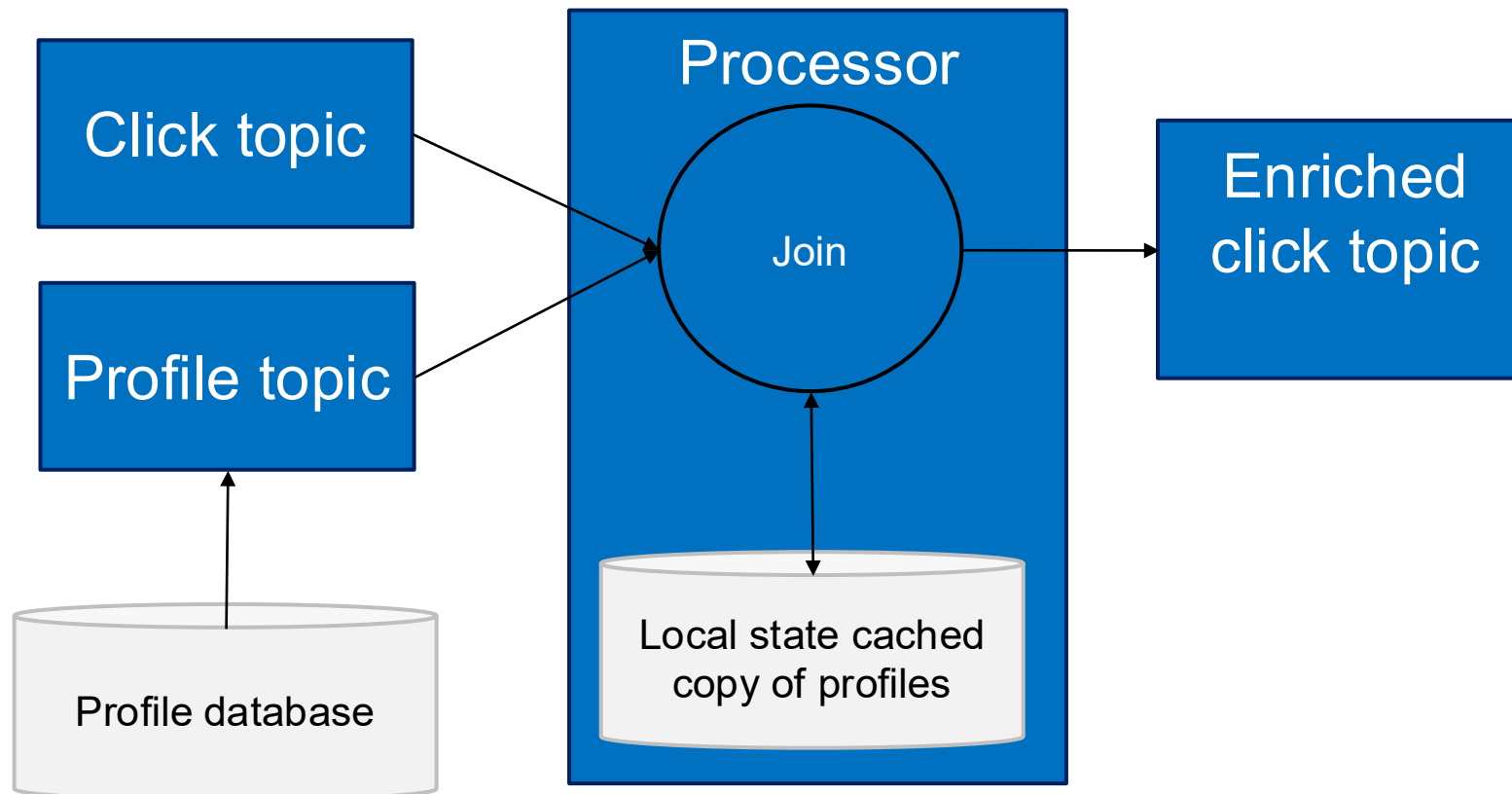
Stateful Processing



Reminder: Event Processing Design Patterns

Processing with External Lookup: Stream-Table Join

Sometimes stream processing requires integration with data external to the stream.

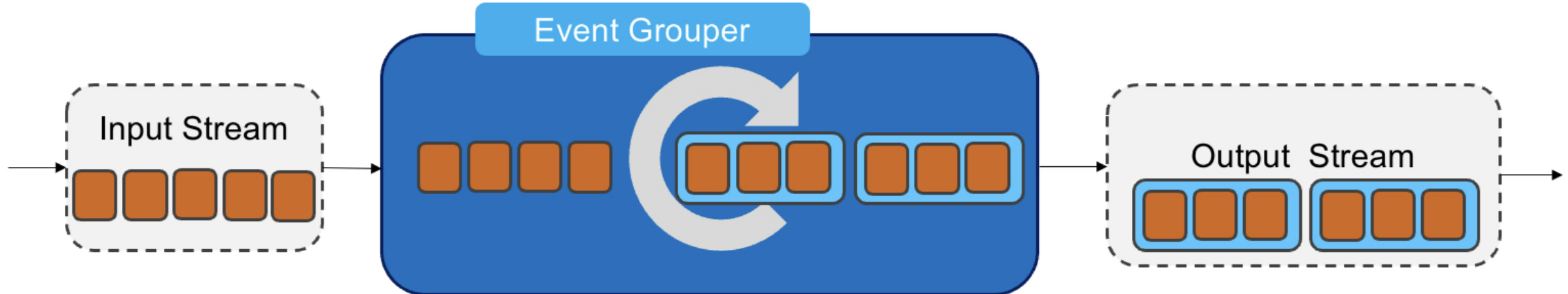


To get good performance and availability, the information from the database needs to be cached in our stream processing application.

Idea: Capturing changes in database table in a stream of events; local database is updated upon changes

Event Grouper. Groups individual but related events from the same event stream (or table)

- Has one input stream / table and one output stream / table
- Groups events together by
 - a common field (*field-based grouping*) (today)
 - and/or by a timestamp (*windowing or time-based grouping*) (next lecture)



Tutorial: How to create tumbling windows with [KStreams](#) and [ksqlDB](#)

Tutorial: How to create a hopping window in [ksqlDB](#)

Grouping Records

Field-based Grouping

- Why grouping?
 - Required before aggregation
 - Ensure related records are processed by the same stream task
 - → Grouping determines how records are organized for aggregation
- How?
 - `groupBy` (may change key → repartitioning)
 - `groupByKey` (uses existing key → preferred)
- Note: Streams versus Tables differ slightly

Grouping Records

Field-based Grouping of Streams

- Grouping is a pre-requisite for aggregating a stream or a table and to ensure that the data is properly partitioned („keyed“), i.e., related records are processed by the same stream task
- Two operators: `groupBy` and `groupByKey`
- → Prefer `groupByKey` if data is already correctly keyed (avoids repartitioning)

```
KGroupedStream<String, Enriched> grouped =  
    withProducts.groupBy(  
        (key, value) -> value.getProductId().toString(),  
        Grouped.with(Serdes.String(), JsonSerdes.Enriched()));
```

KeyValueMapper

KGroupedStream

Grouped

Intermediate representations of a stream which allows you to do aggregations

Grouping Records

Field-based Grouping of Streams

- Grouping is a pre-requisite for aggregating a stream or a table and to ensure that the data is properly partitioned („keyed“), i.e., related records are processed by the same stream task
- Two operators: `groupBy` and `groupByKey`
- → Prefer `groupByKey` if data is already correctly keyed (avoids repartitioning)

```
KGroupedStream<String, Enriched> grouped =  
    withProducts.groupBy(  
        (key, value) -> value.getProductId().toString(),  
        Grouped.with(Serdes.String(), JsonSerdes.Enriched()));
```

Diagram: A box labeled `KeyValueMapper` has a line pointing to the lambda function `(key, value) -> value.getProductId().toString()` in the code above.

```
KGroupedStream<String, Enriched> grouped =  
    withProducts.groupByKey(  
        Grouped.with(Serdes.String(),  
                    JsonSerdes.Enriched()));
```

Diagram: A box labeled `Grouped` has a line pointing to the `Grouped.with` call in the code above. A green checkmark icon is located to the right of the code block.

Already keyed by player ID and
High scores should be calculated by player

Grouping Records

Field-based Grouping of Tables

- Grouping is a pre-requisite for aggregating a stream or a table and to ensure that the data is properly partitioned („keyed“), i.e., related records are processed by the same stream task
- One operators: `groupBy`
- Tables are already keyed → grouping always defines a new key

```
KGroupedTable<String, Player> groupedPlayers =  
    players.groupBy(  
        (key, value) -> KeyValue.pair(key, value),  
        Grouped.with(Serdes.String(), JsonSerdes.Player()) );
```

KeyValueMapper

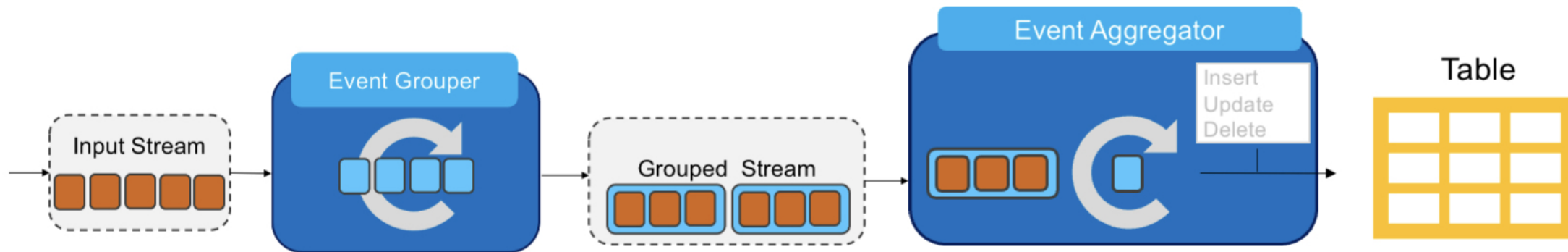
KGroupedTable

Grouped

Intermediate representations of a stream which allows you to do aggregations

Event Aggregator. Combines multiple events into an aggregated results (e.g., totals, averages)

- Has one input stream / table which groups the events (by field and/or by time windows) and one output table (state)
- Result is continuously updated



Related patterns: [Suppressed Event Aggregator](#) (for details see next week)

Details concerning the handling of late events will be discussed next week

How to [sum](#) a stream and how to [count](#) a stream of events

Aggregations in Kafka Streams

Kafka Streams provides three different operators for making aggregations

- `aggregate` → most general (custom logic)
- `reduce` → combine values (output type = input type)
- `count` → counts events per key

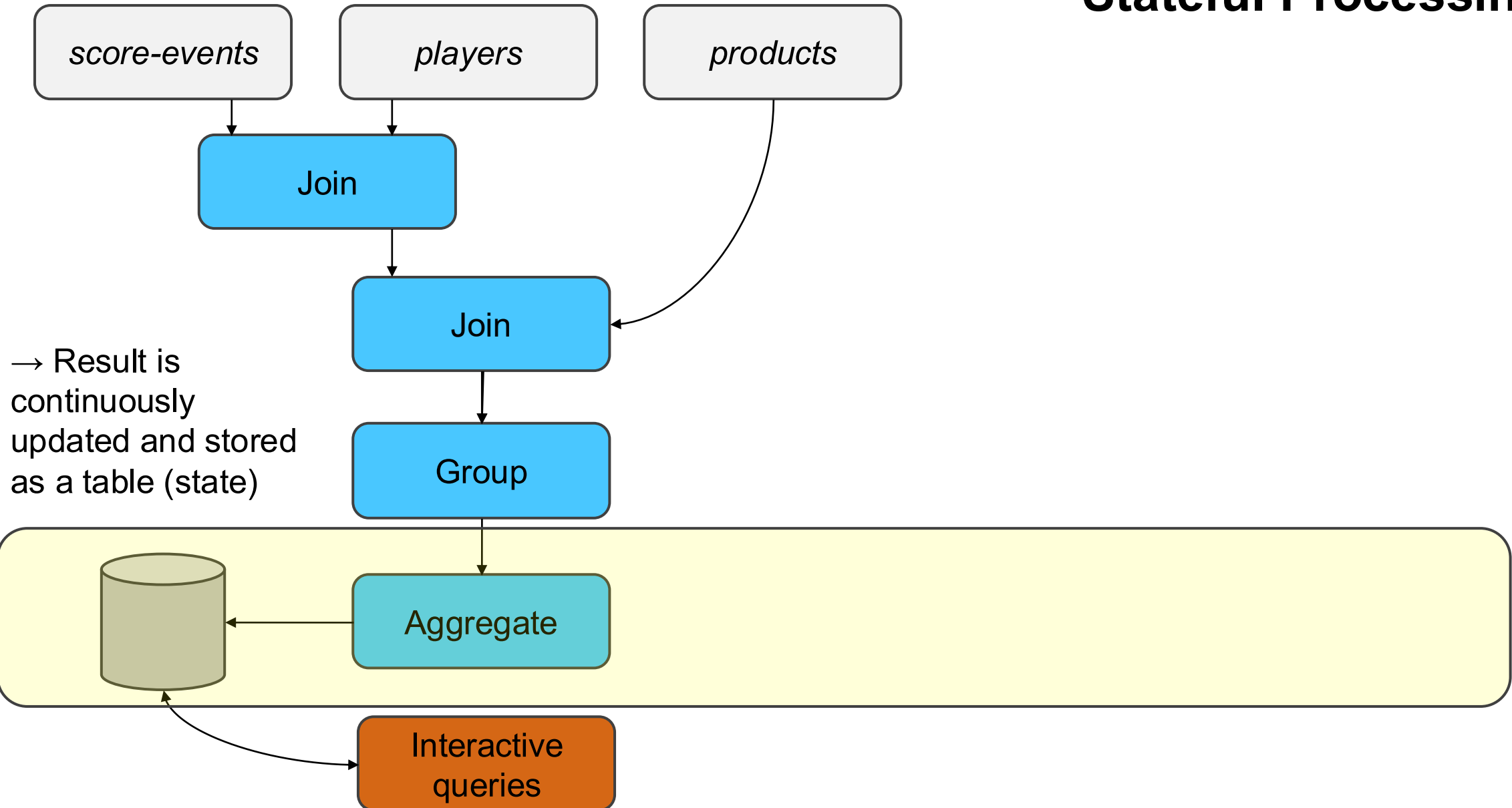
Aggregations in Kafka Streams

- Aggregation = two parts:
 - Initializer → starting value
 - Adder → update state incrementally per event

```
aggregate(  
    () -> new HighScores(),           // initializer  
    (key, value, agg) -> agg.add(value) // adder  
)
```

- The state holds the aggregation result (e.g., count, sum, top-N)

Example Topology Stateful Processing



Aggregations in Kafka Streams

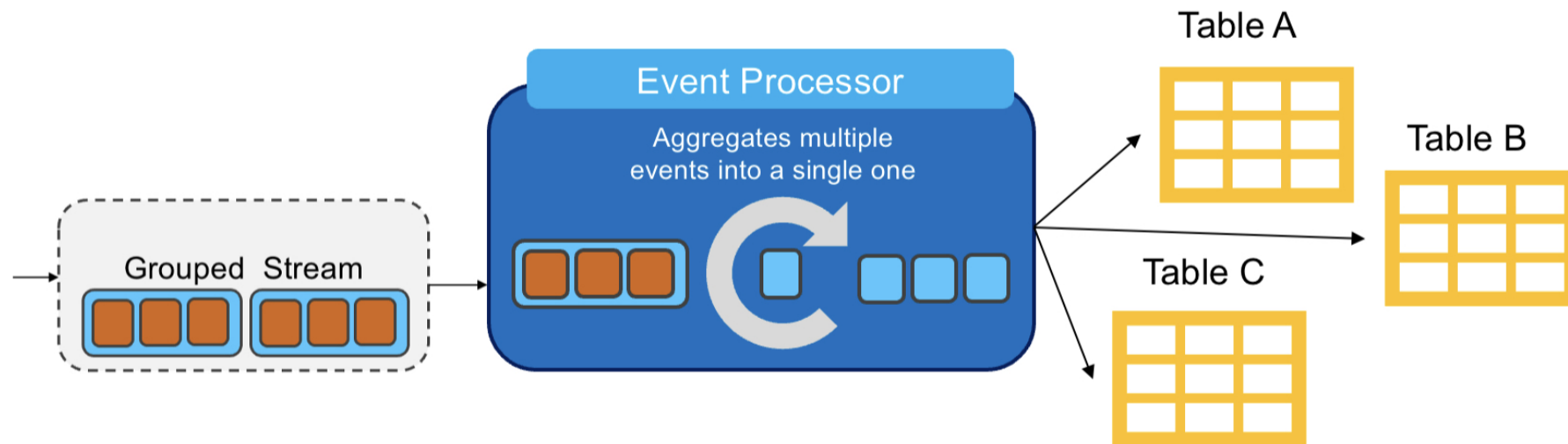
Aggregating streams versus tables

- Streams: initializer and adder (only adding new events)
- Tables: initializer + adder + subtractor (handle updates)

Note: Tables support updates; Streams are append-only

Projection Table. Builds a table (current state) from a stream of events

- Whenever a new event arrives, the table is automatically updated
- To change a projection table, the underlying data needs to be changed by recording new events

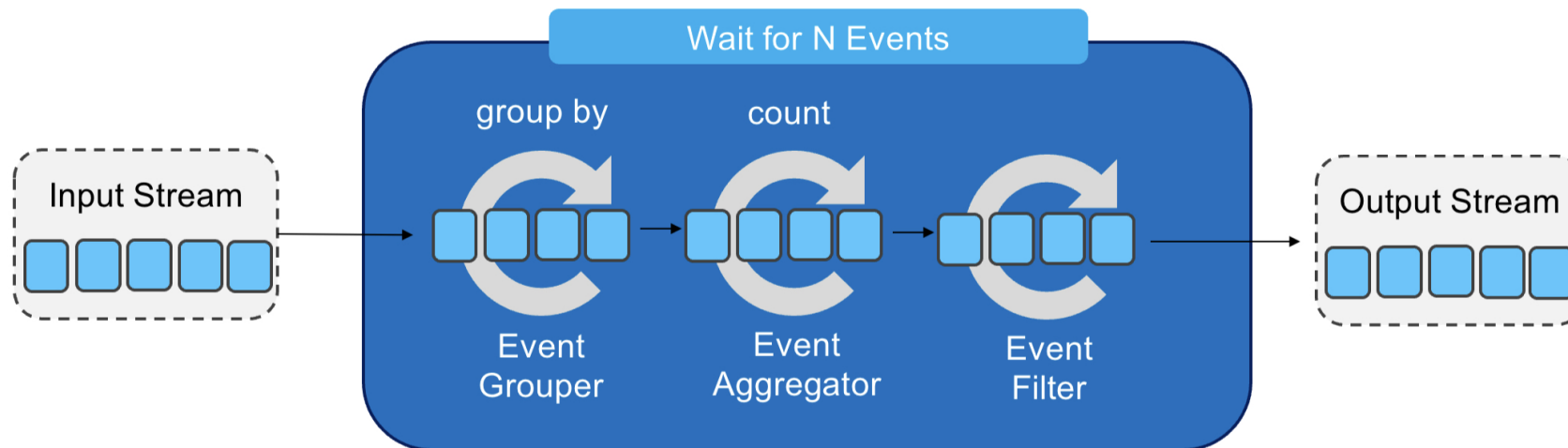


Related patterns: Event Aggregator and State Table

Example Pattern: Wait for N Events

Wait for N Events. Allows to wait for a certain number of events (e.g., a user can try to log in 5 times before the account gets locked)

- Has one input stream / table which groups the events by key and one output stream / table
- Counts the number of occurrences per key
- Filters the output for the relevant count



Related patterns: Event Grouper and Event Aggregator

See also [Wait for N events](#)

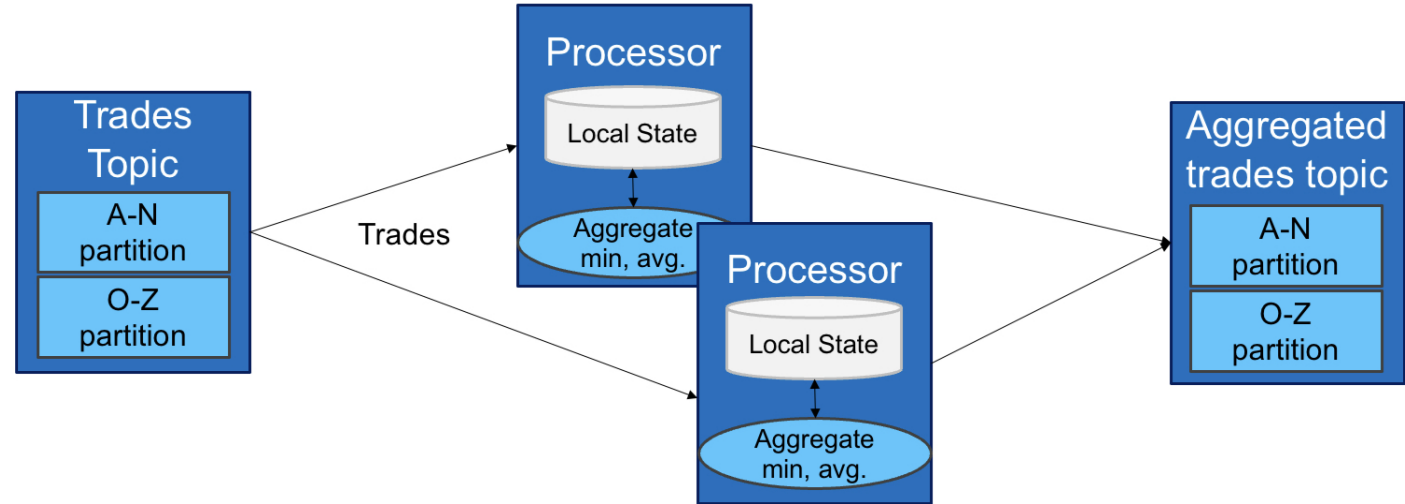
Event Processing Design Patterns

Processing with Local State

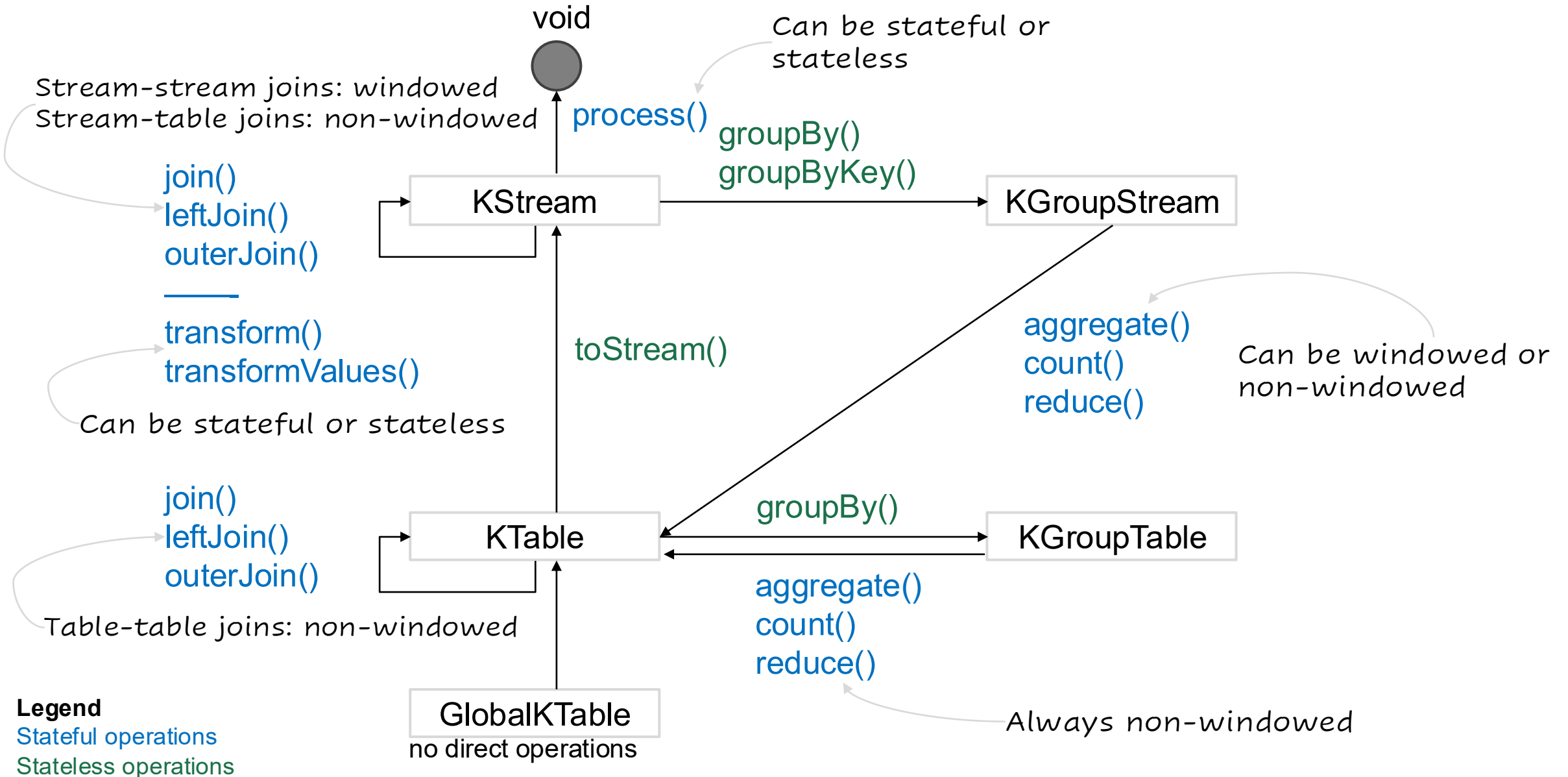
Stateful processing uses local state (state stores)

→ enables aggregation (e.g., min, max, avg)

→ adds complexity: memory, persistence, monitoring



Relationship of Stateful Transformations



Kafka Streams allows exposing application state → locally and externally (via APIs)

The latter makes it easy to build very low latency event-driven microservices

This requires *materializing* the state store.

- Stateful operators like `aggregate`, `reduce`, `count` use **state stores**
- **Internal** state stores
 - only accessed by the processor topology
- **Materialized** state stores:
 - explicitly named
 - queryable from outside the topology
 - materialized = queryable state
- To enable read-only access of the state, the store must be materialized using an overloaded method

Materialized Stores

```
Initializer<HighScores> highScoresInitializer = HighScores::new;  
Aggregator<String, Enriched, HighScores> highScoresAdder =  
    (key, value, aggregate) -> aggregate.add(value);  
KTable<String, HighScores> highScores =  
    grouped.aggregate(highScoresInitializer, highScoresAdder);
```

Internal State
Store

```
Initializer<HighScores> highScoresInitializer = HighScores::new;  
Aggregator<String, Enriched, HighScores> highScoresAdder =  
    (key, value, aggregate) -> aggregate.add(value);  
KTable<String, HighScores> highScores =  
    grouped.aggregate(  
        highScoresInitializer,  
        highScoresAdder,  
        Materialized.<String, HighScores, KeyValueStore<Bytes, byte[]>>  
            as("leader-boards")  
            .withKeySerde(Serdes.String())  
            .withValueSerde(JsonSerdes.HighScores()));
```

Only important:
→ naming the store
makes it queryable

Materialized.as
method includes
three generics: key
type, value type, and
type of state store

Store is explicitly named to make it
available for querying outside of the
processor topology

Accessing Read Only Stores

- To access a state store you need
 - Store name
 - Store type
- Use `QueryableStoreTypes` to get the correct read-only store

```
ReadOnlyKeyValueStore<String, HighScores> stateStore =  
    streams.store(  
        StoreQueryParameters.fromNameAndType(  
            "leader-boards",  
            QueryableStoreTypes.keyValueStore()));
```

Querying Nonwindowed Key-Value Stores

- Depending on the state store type different queries are supported

```
public interface ReadOnlyKeyValueStore<K, V> {
```

```
    V get(K key);
```

Point lookups: return value for given key

```
    KeyValueIterator<K, V> range(K from, K to);
```

Range scan: returns iterator for inclusive range of keys; to avoid memory leaks closing the iteration is crucial

```
    KeyValueIterator<K, V> all();
```

All entries: returns an iterator with all key-value pairs;

```
    long approximateNumEntries();
```

```
}
```

Number of entries: returns the approximate number of keys in the underlying state store; to obtain an exact number use an in-memory store

Local Queries vs. Remote Queries

- Local queries
 - Each instance can query its own **local state**
 - **Local state** = **partial view** of the full application state
 - Remote queries
 - Query the full application state accross instances
- **Trade-off:** local = fast, remote = complete

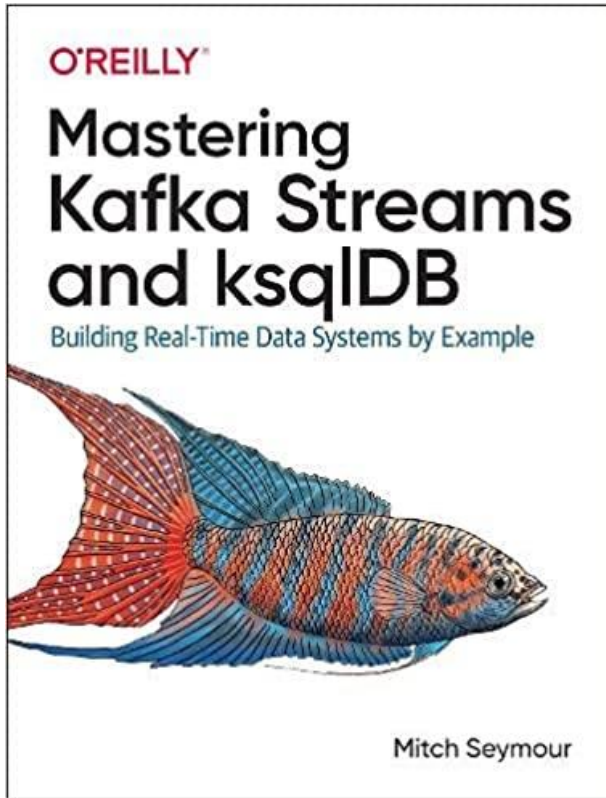
- **Remote queries:** → query state across instances
- **Requires**
 - Locating the right instance
 - Exposing state via RPC / REST

Event Processing Design Patterns

Interactive Queries

Traditional → read results from output topics

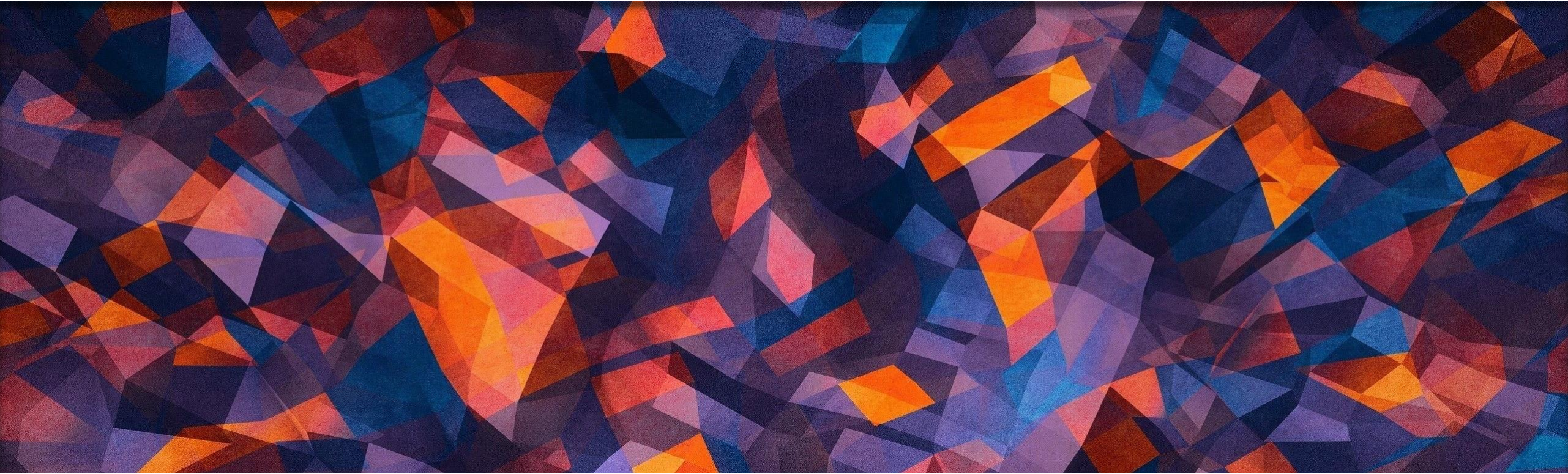
With interactive queries → read state directly from the application



Event Streaming Patterns

<https://developer.confluent.io/patterns>

[Repository and book chapters](#)

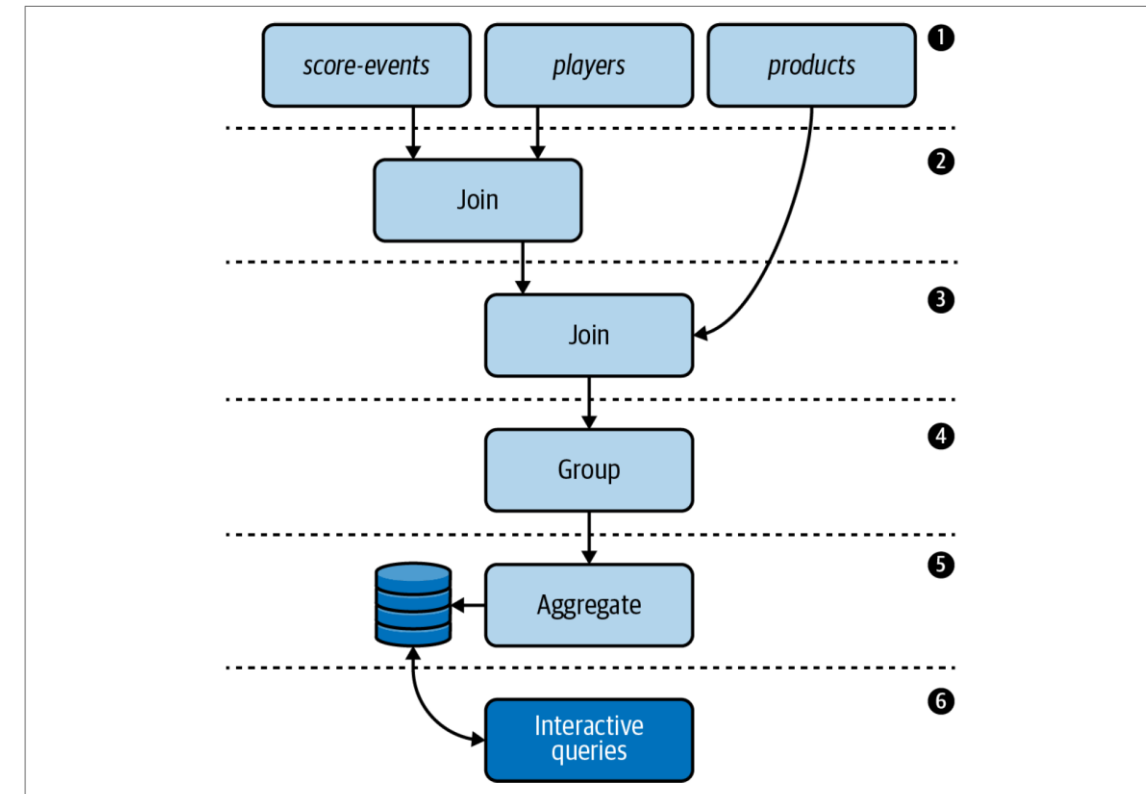


Event-driven and Process-oriented Architectures

Exercises

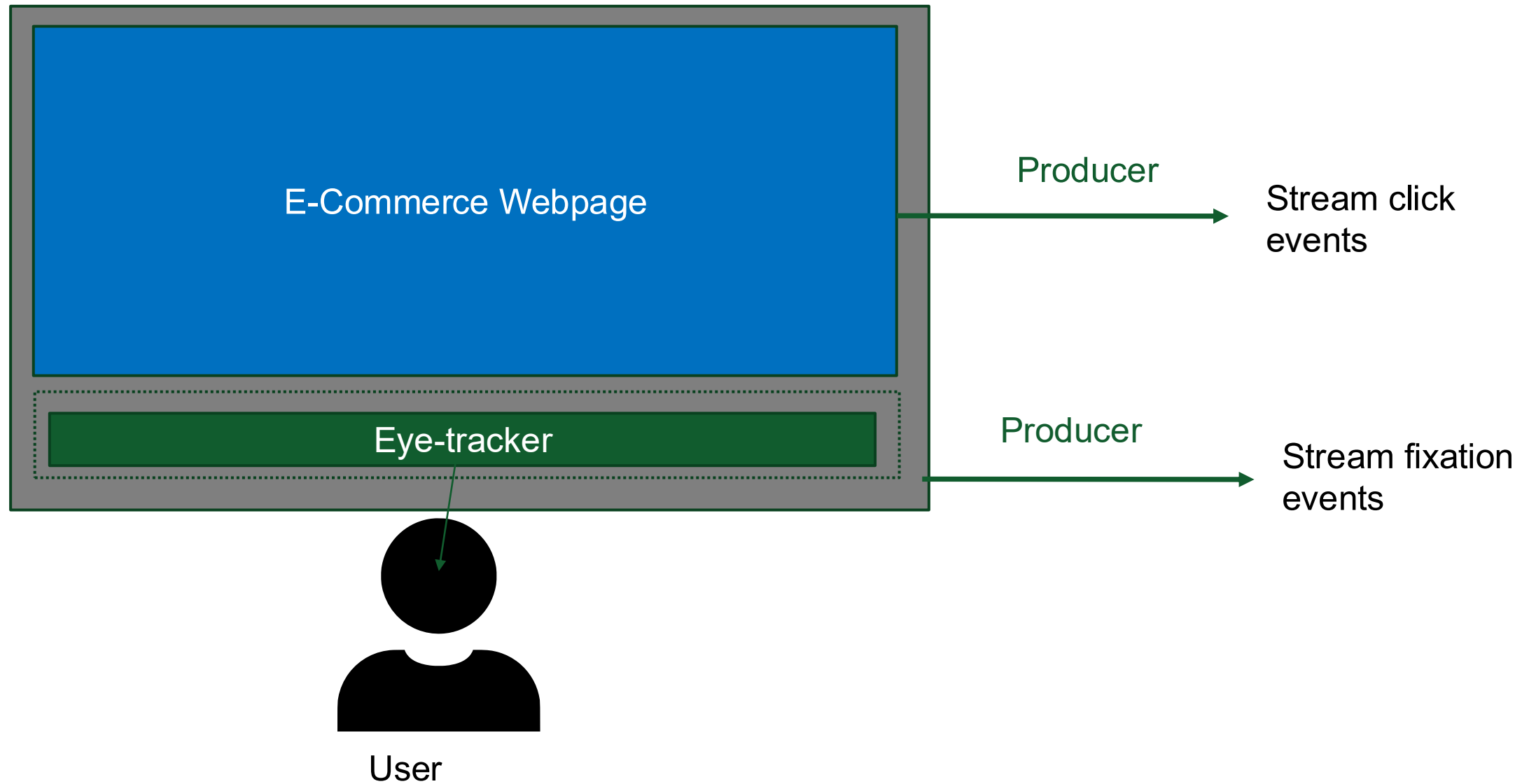
Lecture 9

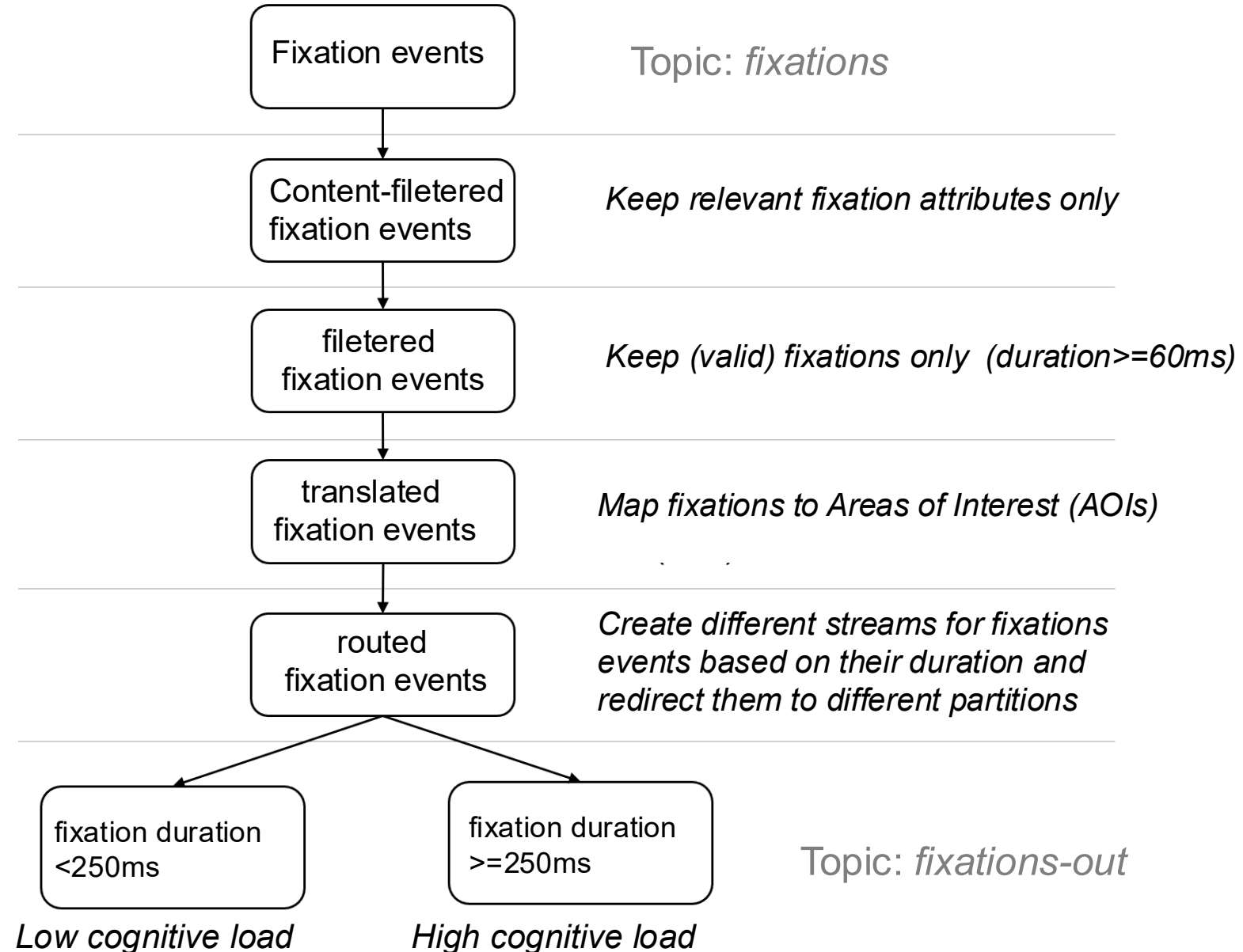
- For Kafka Streams stateful (part 1):
 - Leaderboard App:
 - <https://github.com/scs-edpo/lab13Part1-kafka-streams-leaderboard>
 - Chapter 4 in: Mastering Kstreams and ksqlDB



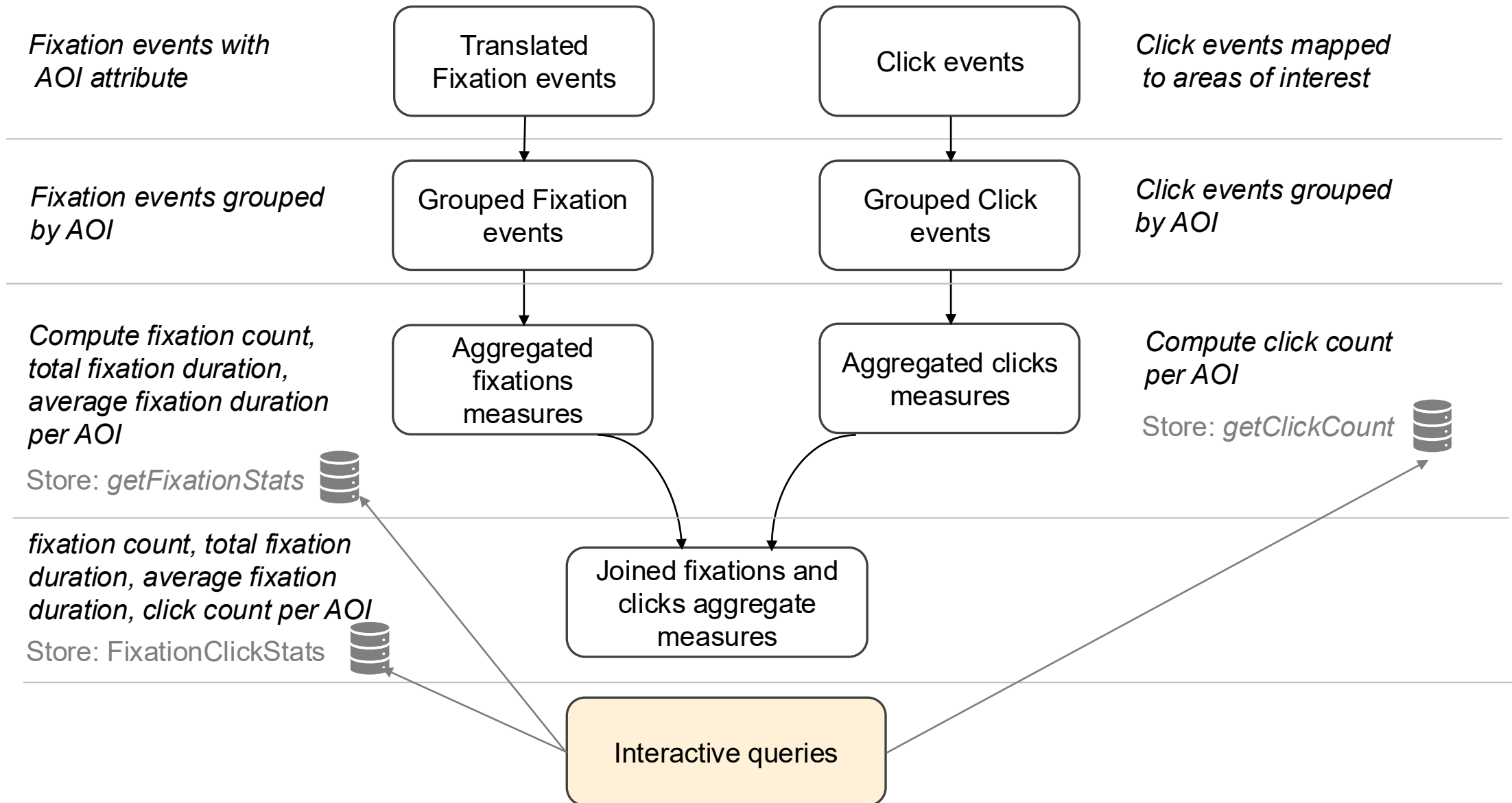
- For Kafka Streams stateless and stateful event processing in the eye-tracking domain (part 2):
 - Eye-Tracking Events Processing App:
 - <https://github.com/scs-edpo/lab13Part2-kafka-streams-eyeTracking2>

Computer screen



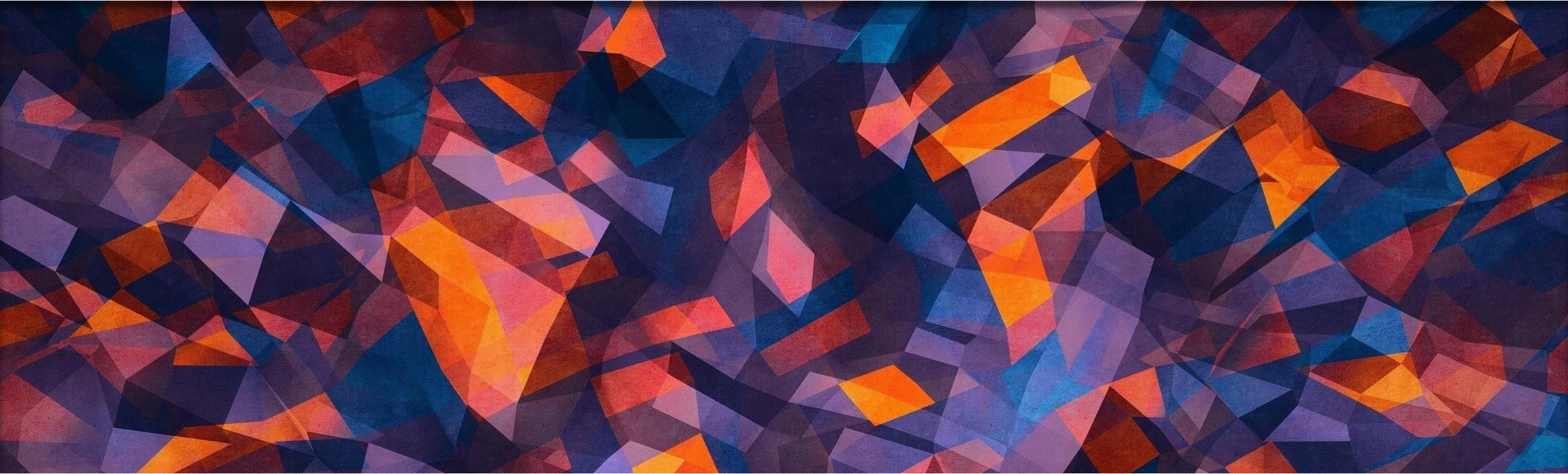


Topic: *clicks*





Universität St.Gallen



Event-driven and Process-oriented Architectures

Project

Lecture 9

1. Develop one or several stream processing applications
2. The stream processing applications should include both stateless and stateful stream processing
 - At least several stateless operations (Week 8)
 - Usage of both streams and tables (Week 9)
 - Consider data from more than one stream (Week 9)
 - Interactive queries (Week 9)
 - Windowed operations (Week 10)
3. Describe for each developed stream processing application the topology in graphical form
4. Elaborate on the chosen patterns
5. Use Avro schema (registryless or with a schema registry)

